

Analisi e Progettazione del Software

Diagrammi di interazione

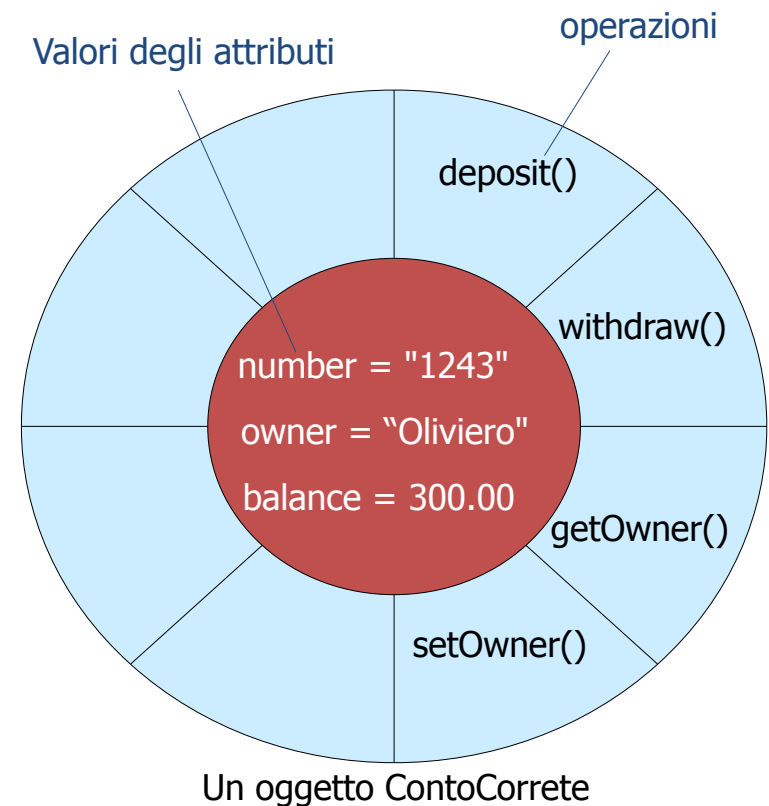
Oliviero Riganelli

Che cosa sono gli oggetti

- Un oggetto è un pacchetto coeso di dati e funzioni incapsulate in una unità riusabile. Oggetti incapsulare i dati.
- Ogni oggetto è un'istanza di una classe che definisce l'insieme comune di caratteristiche (attributi e operazioni) condivisi da tutte le sue istanze. Gli oggetti hanno:
 - *I valori degli attributi - la parte dei dati*
 - *Le operazioni - la parte comportamento*
- Tutti gli oggetti hanno:
 - *Identità: ogni oggetto ha il suo identificativo univoco*
 - *Stato: viene stabilito dai valori effettivi dei dati memorizzati in un oggetto ad un dato istante*
 - *Comportamento: l'insieme delle operazioni che l'oggetto può eseguire*

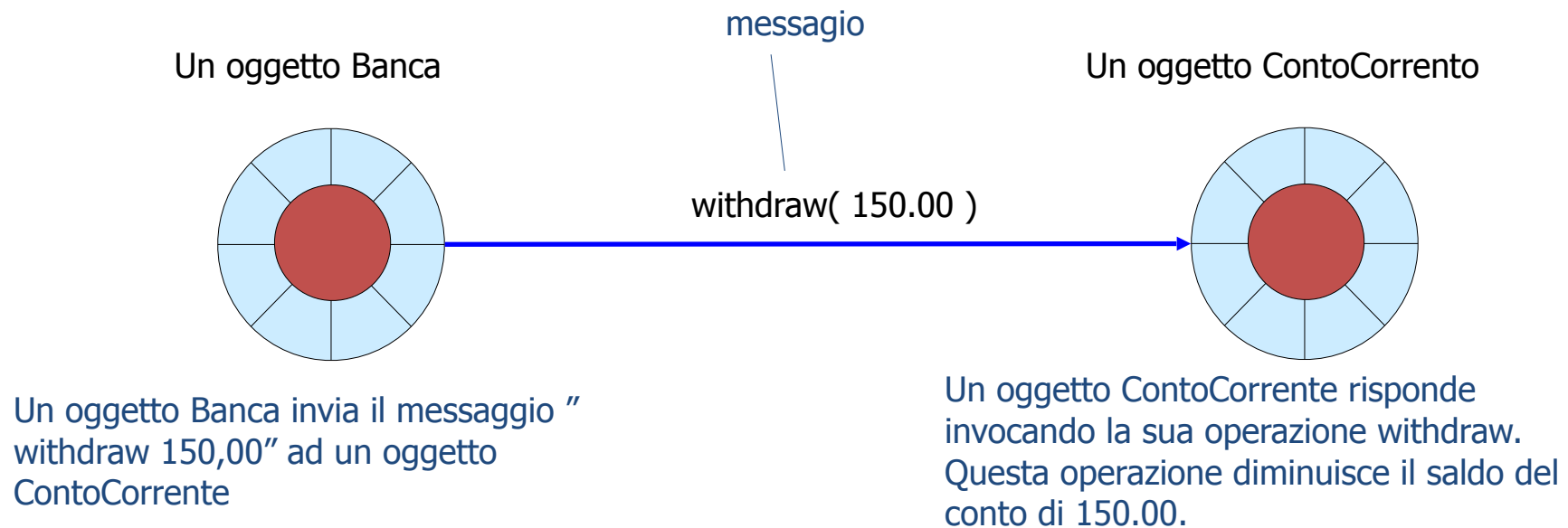
Incapsulamento

- I dati sono nascosti all'interno dell'oggetto, l'unico modo per accedere ai dati è attraverso le operazioni
- **incapsulamento** o **data hiding**:
 - *Software più robusto*
 - *Codice riutilizzabile*



Messaggi

- Gli oggetti collaborano per eseguire le funzioni del sistema stabilendo dei collegamenti tra loro e scambiandosi messaggi
- I messaggi causano l'invocazione delle operazioni da parte dell'oggetto



Diagrammi di interazione

- In UML i **digrammi di interazione** illustrano il modo in cui gli oggetti interagiscono attraverso lo scambio di messaggi
- Sono utilizzati per la **modellazione dinamici degli oggetti**

Interazioni

- Un'interazione è una specifica di come alcuni oggetti si scambiano messaggi nel tempo per eseguire un compito nell'ambito di un certo contesto
- I diagrammi di interazione catturano un'interazione come
 - *Linee di vita – partecipanti nell'interazione*
 - *Messaggi – comunicazioni tra linee di vita*

Linee di vita

- Una linea di vita rappresenta un singolo partecipante a un'interazione
 - *Raffigura come un'istanza del classificatore partecipa all'interazione*
- Le linee di vita hanno:
 - **Nome:** *usato per far riferimento alla line di vita nell'interazione*
 - **Tipo:** *il nome del classificatore di cui rappresenta un'istanza*
 - **Selettore:** *una condizione booleana che selezione una specifica istanza*
- Le linee di vita sono disegnate con la stessa icona del loro classificatore
 - *La linea di vita promessiSposi rappresenta un'istanza della classe Libro*
 - *il selettore [id="1234"] seleziona una specifica istanza di Libro con id "1234"*

promessiSposi [id = "1234"] : Libro

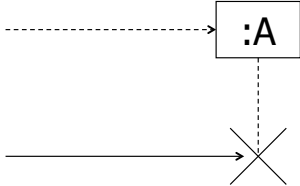
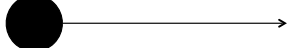
nome

selettore

tipo

Messaggi

Un messaggio rappresenta la comunicazione tra due linee vita

Sintassi	Nome	Semantica
	Messaggio sincrono	Il mittente aspetta che il destinatario ritorni dall'esecuzione del messaggio
	Messaggio asincrono	Il mittente invia il messaggio e continua l'esecuzione: non aspetta un Ritorno dal destinatario
	Messaggio di ritorno	Il destinatario di un messaggio precedente restituisce il focus di Controllo al mittente di quel messaggio
	Creazione dell'oggetto Distruzione dell'oggetto	Il mittente crea un'istanza del classificatore specificato dal destinatario Il mittente distrugge il destinatario. Se la sua linea di vita ha una coda, questa termina con una X
	Messaggio trovato	il messaggio viene inviato dal di fuori dell'ambito dell'interazione. Si vuole mostrare la ricezione del messaggio ma non la provenienza.
	Messaggio perso	Il messaggio non raggiunge mai la sua destinazione. Si può usare per indicare condizioni di errore in cui i messaggi vanno persi.

Diagrammi di interazione

- **Diagrammi di sequenza**

- *Enfatizzano la sequenza temporale degli scambi di messaggi*
- *Mostrano interazioni ordinate in una sequenza temporale*
- *Non mostrano le relazioni degli oggetti (possono essere dedotte dagli invii di messaggi)*

- **Diagrammi di comunicazione**

- *Enfatizzano le relazioni strutturali tra gli oggetti*
- *Sono utilizzati per esplicitare le relazioni degli oggetti*

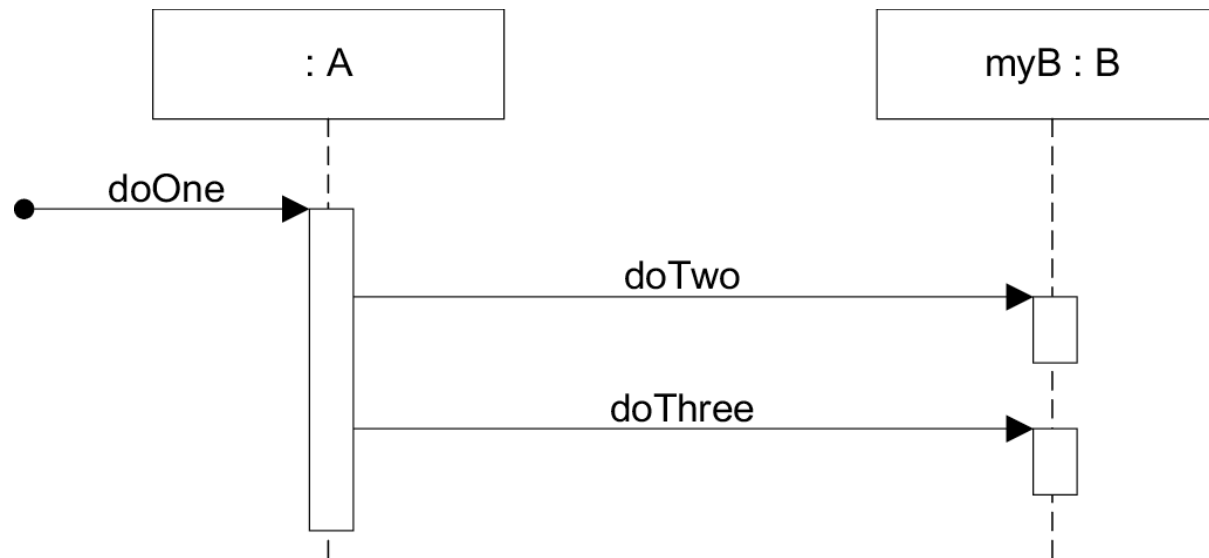
- **Diagrammi di interazione generale**

- *Illustrano come il comportamento complesso è realizzato da una serie di interazioni più semplici*

- **Diagrammi di temporizzazione**

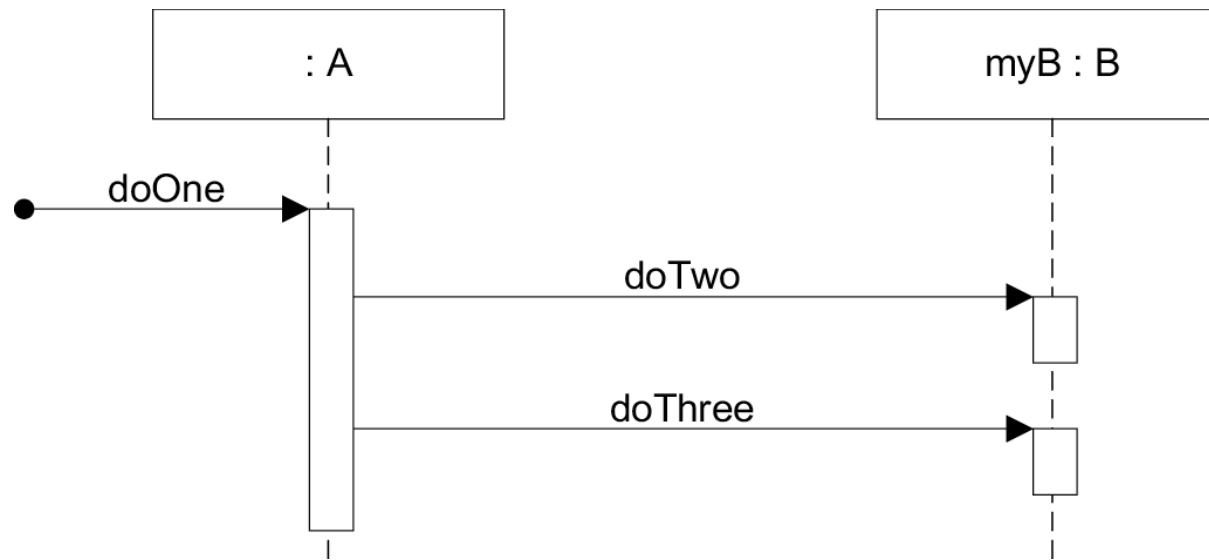
- *Enfatizzano gli aspetti in tempo reale di una interazione*

Diagramma di sequenza

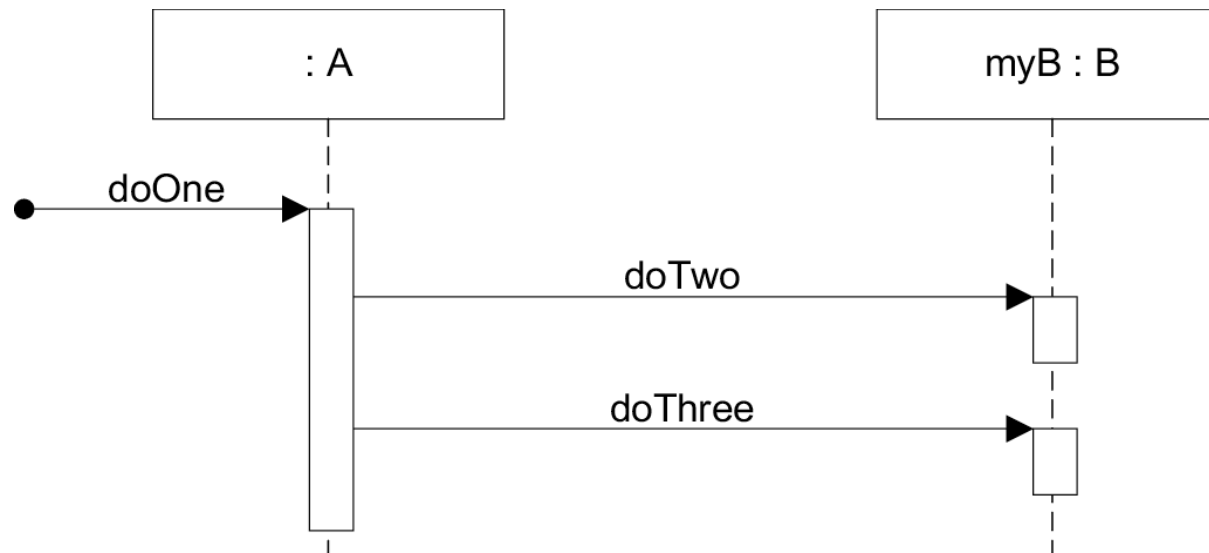


- Un diagramma di sequenza mostra l'interazione tra un insieme di oggetti
 - *formato a steccato*

Cosa vuol dire questo diagramma nel codice?



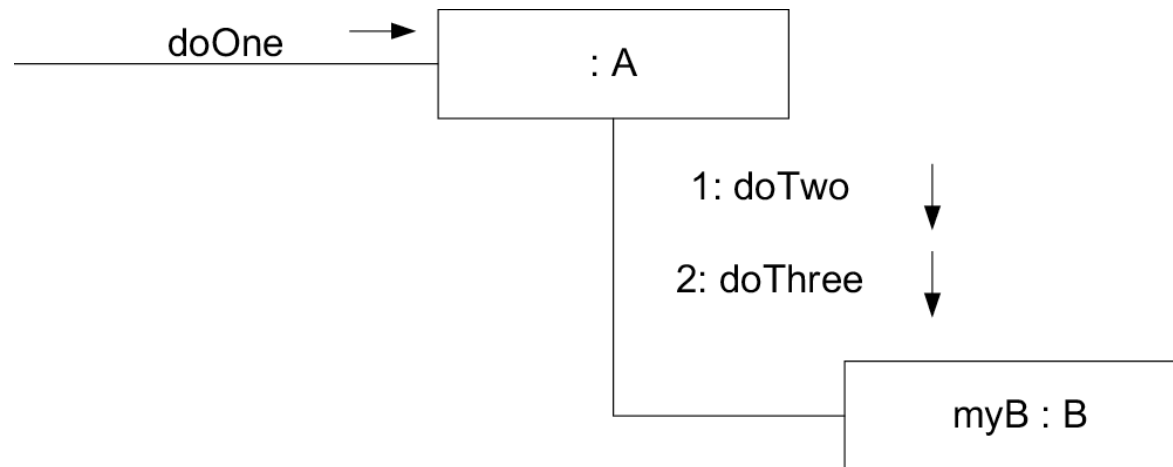
Cosa vuol dire questo diagramma nel codice?



```
public class A {
    private B myB = new B();
    public void doOne() {
        myB.doTwo();
        myB.doThree();
    }
    ...
}
```

```
public class B {
    public void doTwo() {
        ...
    }
    public void doThree() {
        ...
    }
    ...
}
```

Diagramma di comunicazione

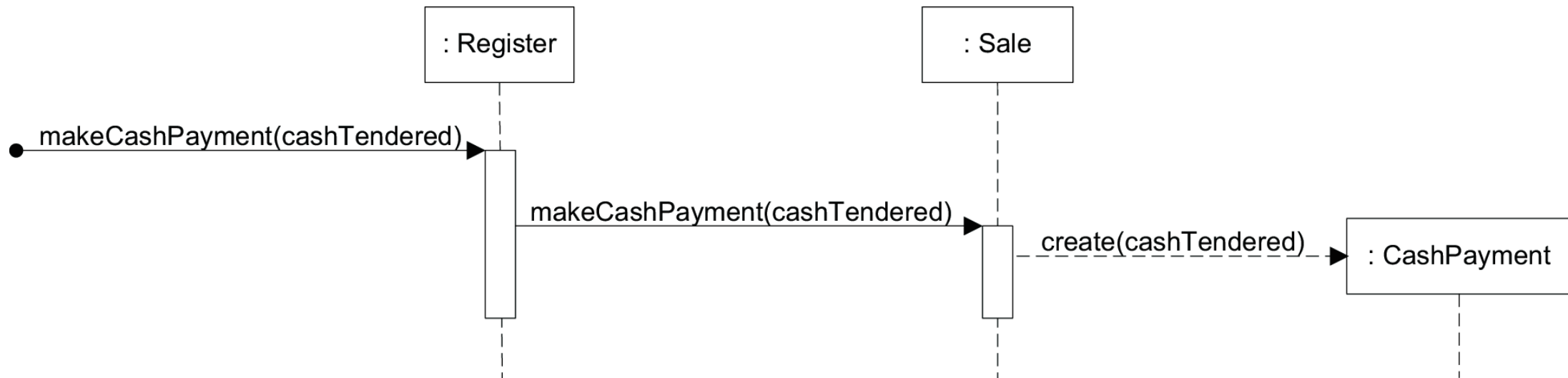


- Un diagramma di comunicazione mostra l'interazione tra un insieme di oggetti
 - *formato a grafo o a rete*

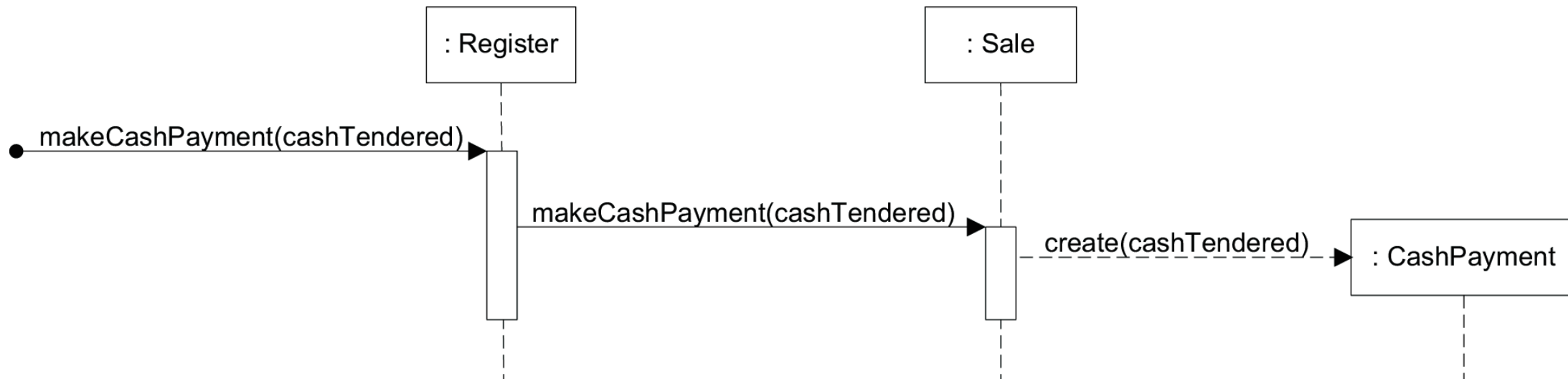
Punti di forza e debolezza dei diagrammi di sequenza e di comunicazione

Tipo	Punti di forza	Punti di debolezza
Diagramma di sequenza	Mostra chiaramente la sequenza o l'ordinamento temporale dei messaggi. Opzioni di notazione numerose e dettagliate.	Costringe a estendersi verso destra quando si aggiungono nuovi oggetti; consuma lo spazio orizzontale.
Diagramma di comunicazione	Economizza lo spazio; flessibilità nell'aggiunta di nuovi oggetti nelle due dimensioni.	Più difficile vedere la sequenza dei messaggi. Meno opzioni di notazione.

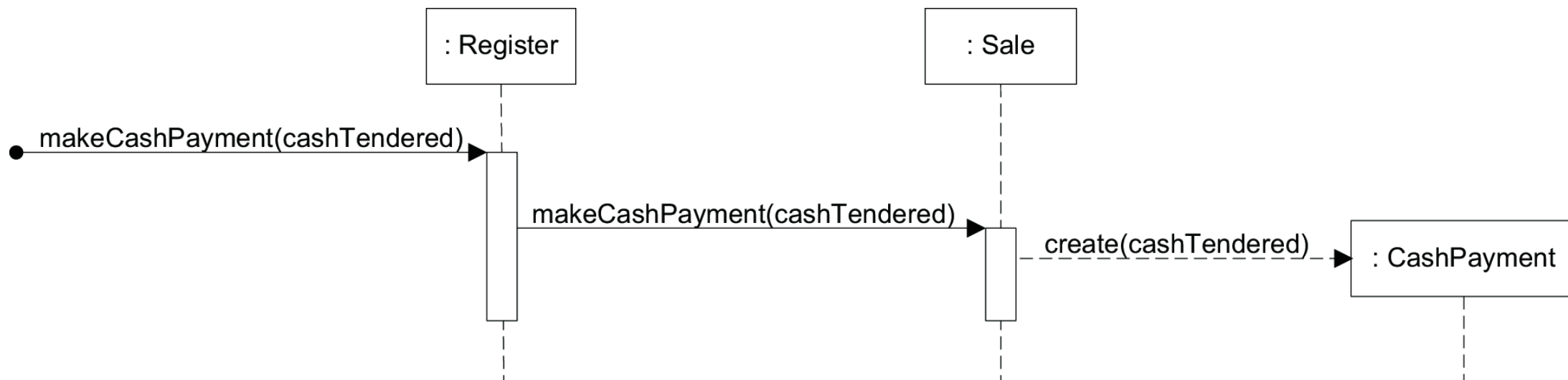
Esempio di diagramma di sequenza: makeCashPayment



Quale potrebbero essere il codice corrispondente alla classe *Register* e nel suo metodo *makeCashPayment*?

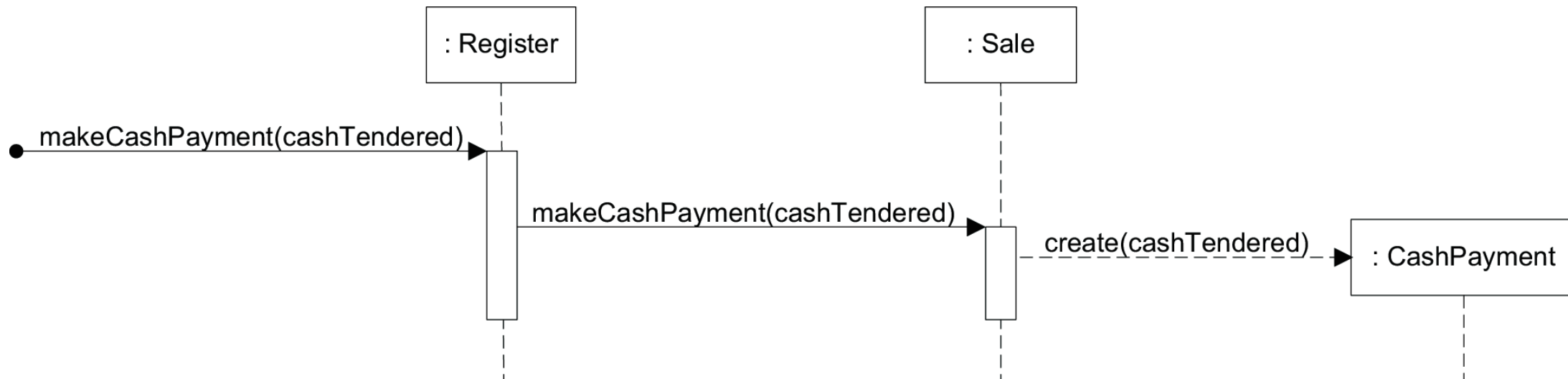


Quale potrebbero essere il codice corrispondente alla classe *Register* e nel suo metodo *makeCashPayment*?

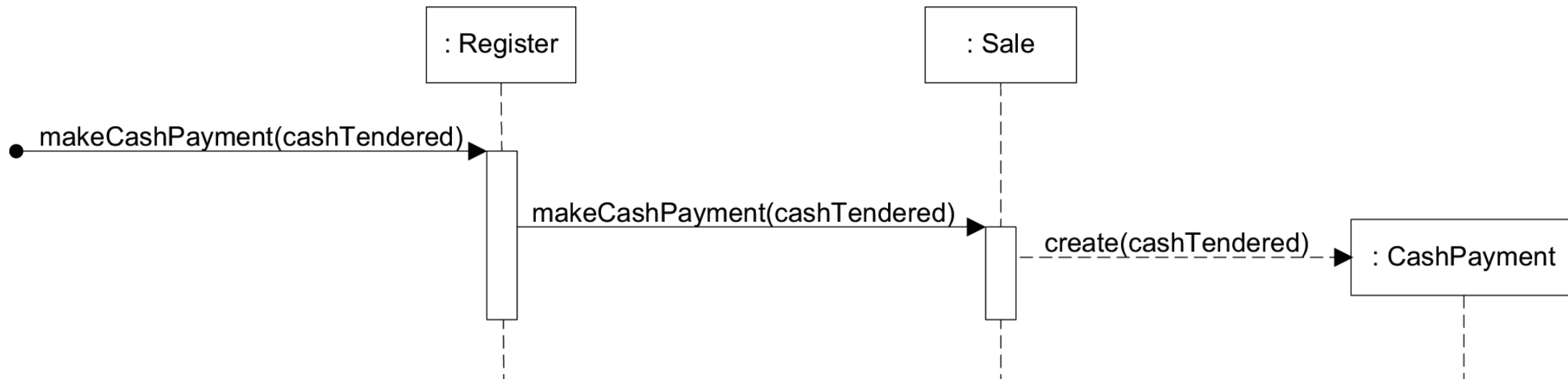


```
public class Register {
    private Sale currentSale;
    public void makeCashPayment(Money cashTendered) {
        currentSale.makePayment(cashTendered);
        ...
    }
    ...
}
```

Quale potrebbero essere il codice corrispondente alla classe *Sale* e nel suo metodo *makeCashPayment*?

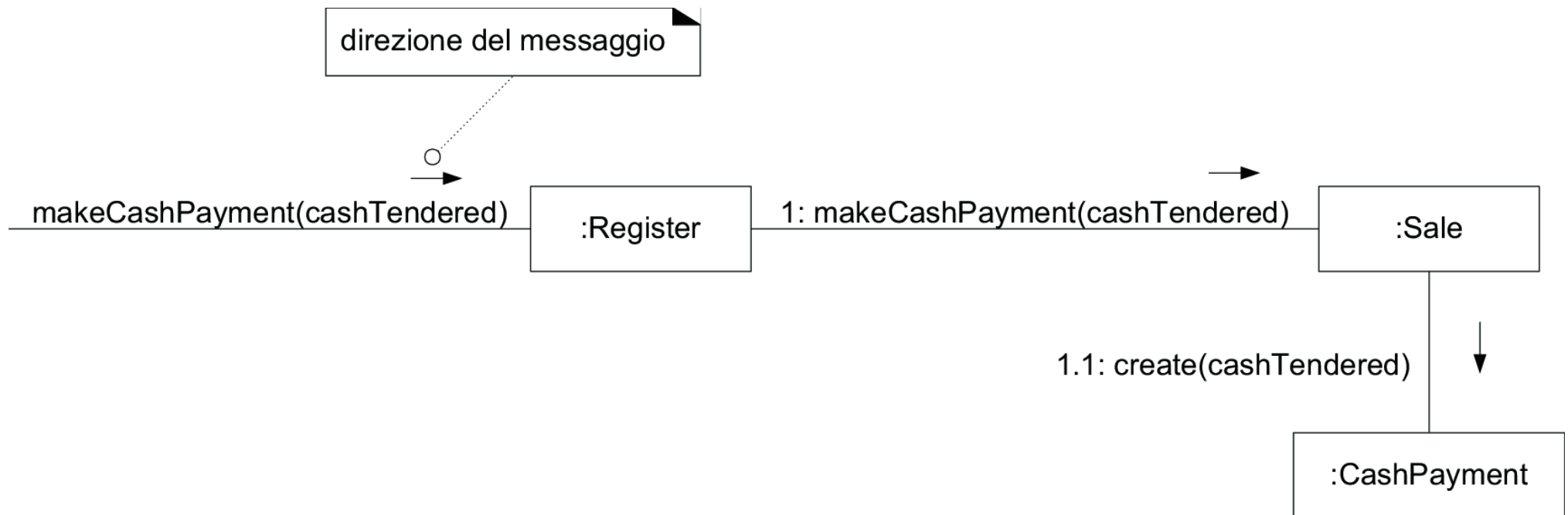


Quale potrebbero essere il codice corrispondente alla classe *Sale* e nel suo metodo *makeCashPayment*?



```
public class Sale {
    private CashPayment payment;
    public void makeCashPayment(Money cashTendered) {
        payment = new CashPayment(cashTendered);
        ...
    }
    ...
}
```

Esempio di diagramma di comunicazione: makeCashPayment

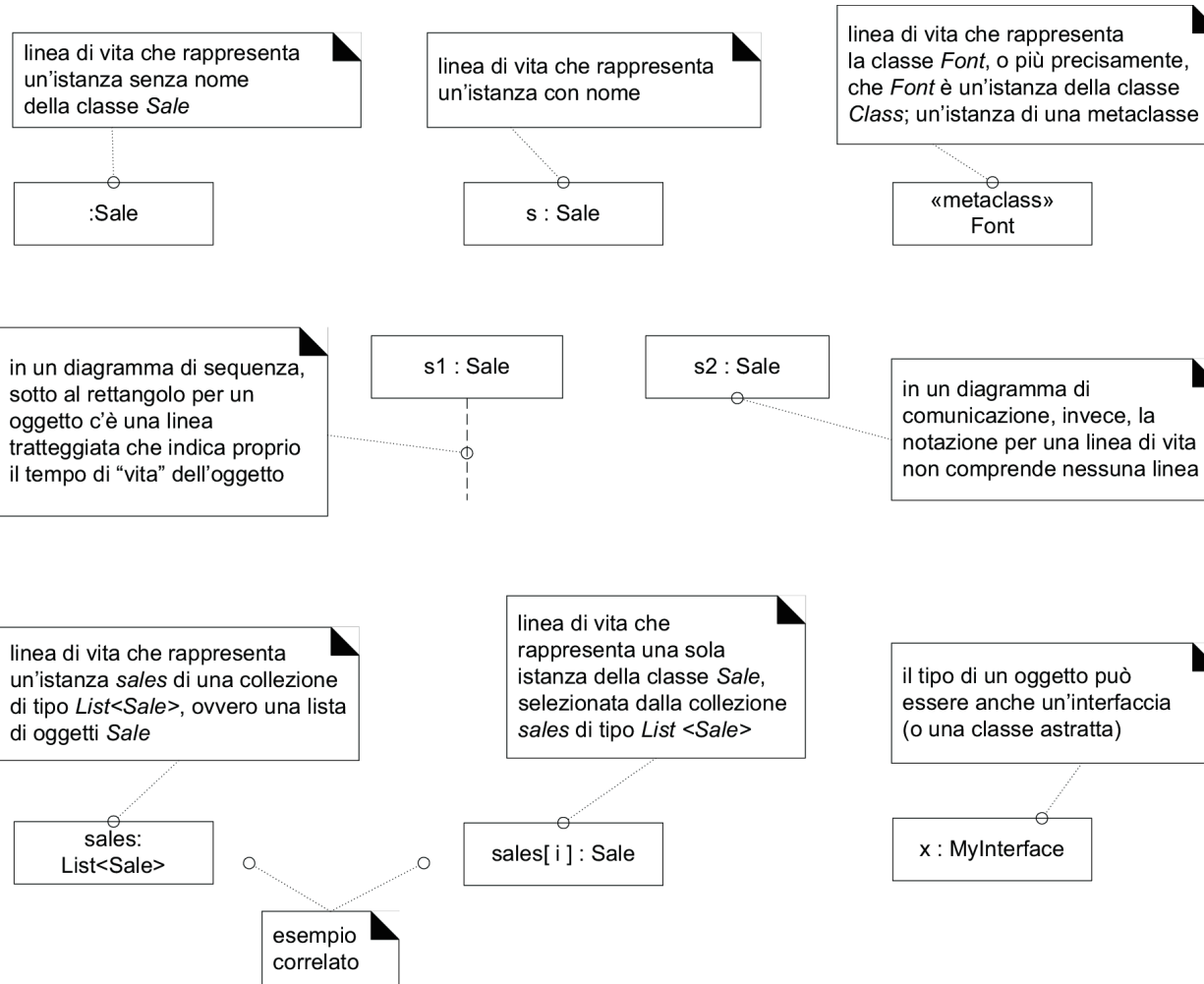


Spesso non si presta sufficiente attenzione ai diagrammi di interazione

- Modellazione durante la progettazione
 - *aspetti statici – diagrammi delle classi*
 - *aspetti dinamici – diagrammi di interazione*
- Linea guida
 - *Si dedichi del tempo alla modellazione dinamica degli oggetti e non solo alla modellazione statica degli oggetti con i diagrammi delle classi*
- Suggerimento
 - *Creare questi modelli in parallelo*

Notazione comune dei diagrammi di interazione:

Linee di vita



Notazione comune dei diagrammi di interazione: espressioni messaggio

Gli oggetti interagiscono scambiandosi messaggi

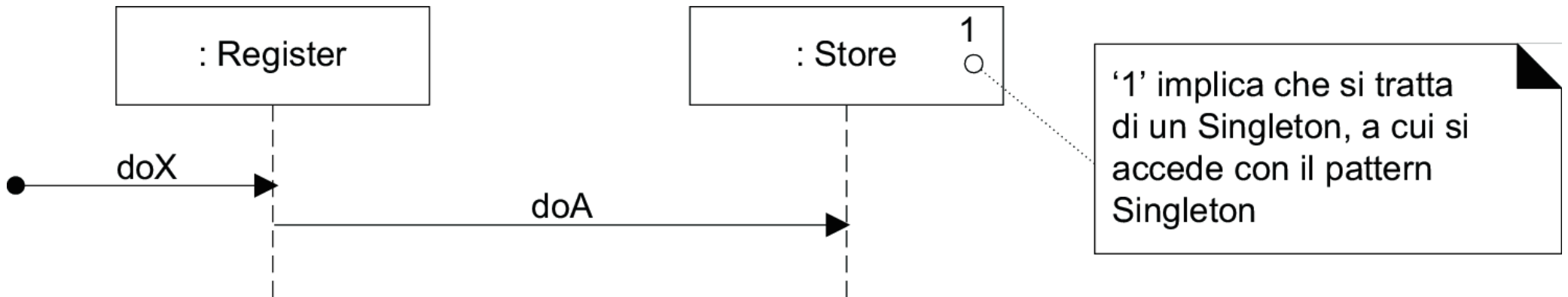
- UML ha una sintassi std per queste espressioni

```
return = message(parameter : parameterType) : returnType
```

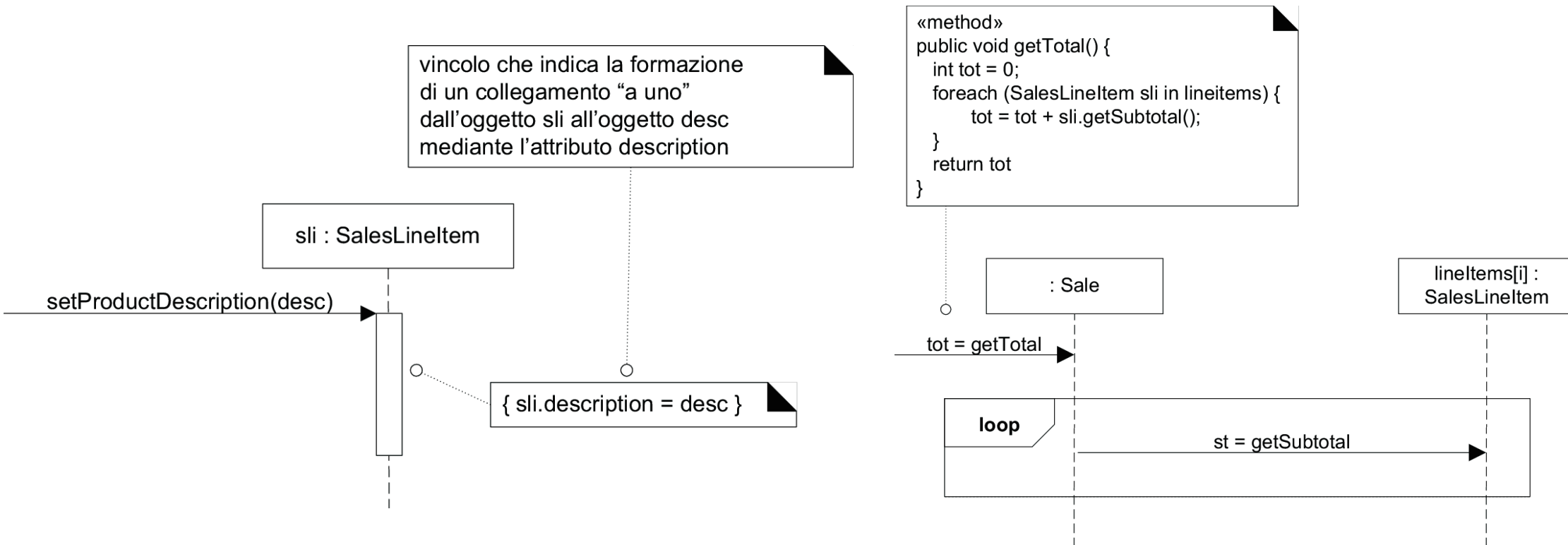
Tutti i seguenti esempi sono validi, ma descrivono informazioni e dettagli diversi

```
initialize(code)  
initialize  
d = getProductDescription(id)  
d = getProductDescription(id:ItemID)  
d = getProductDescription(id:ItemID) : ProductDescription
```

Notazione comune dei diagrammi di interazione: Oggetto Singleton



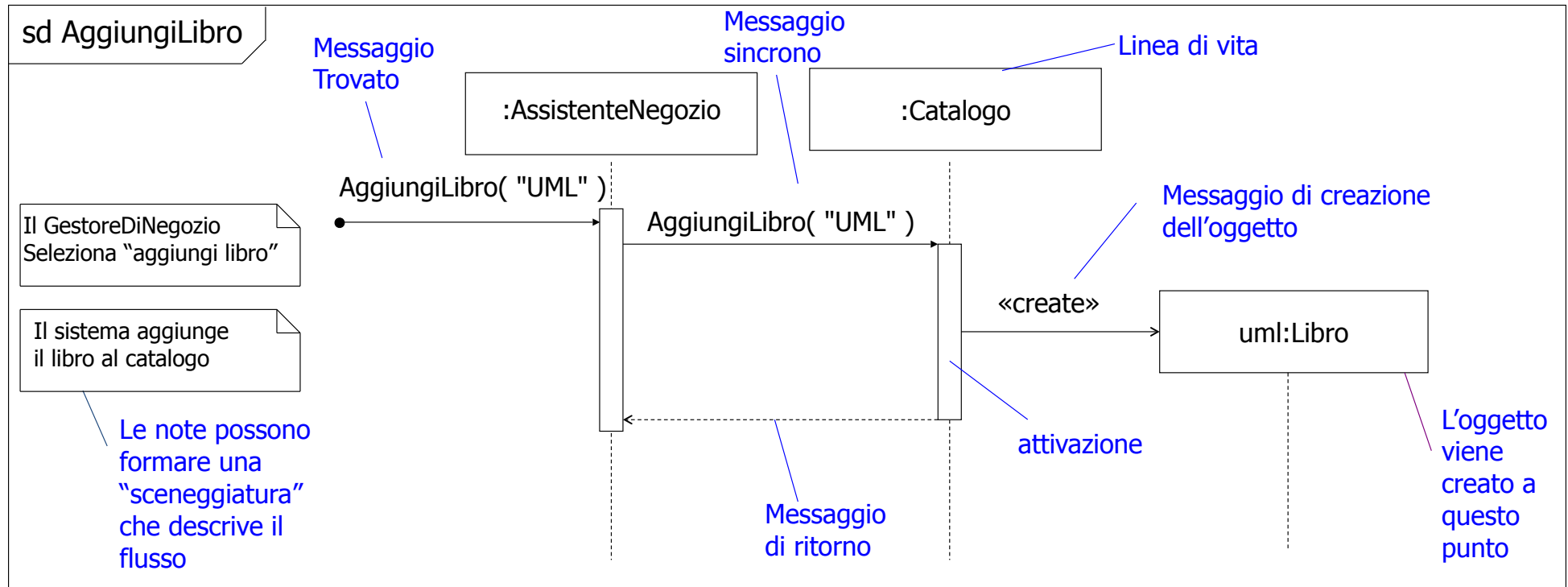
Notazione comune dei diagrammi di interazione: Note



Vincolo

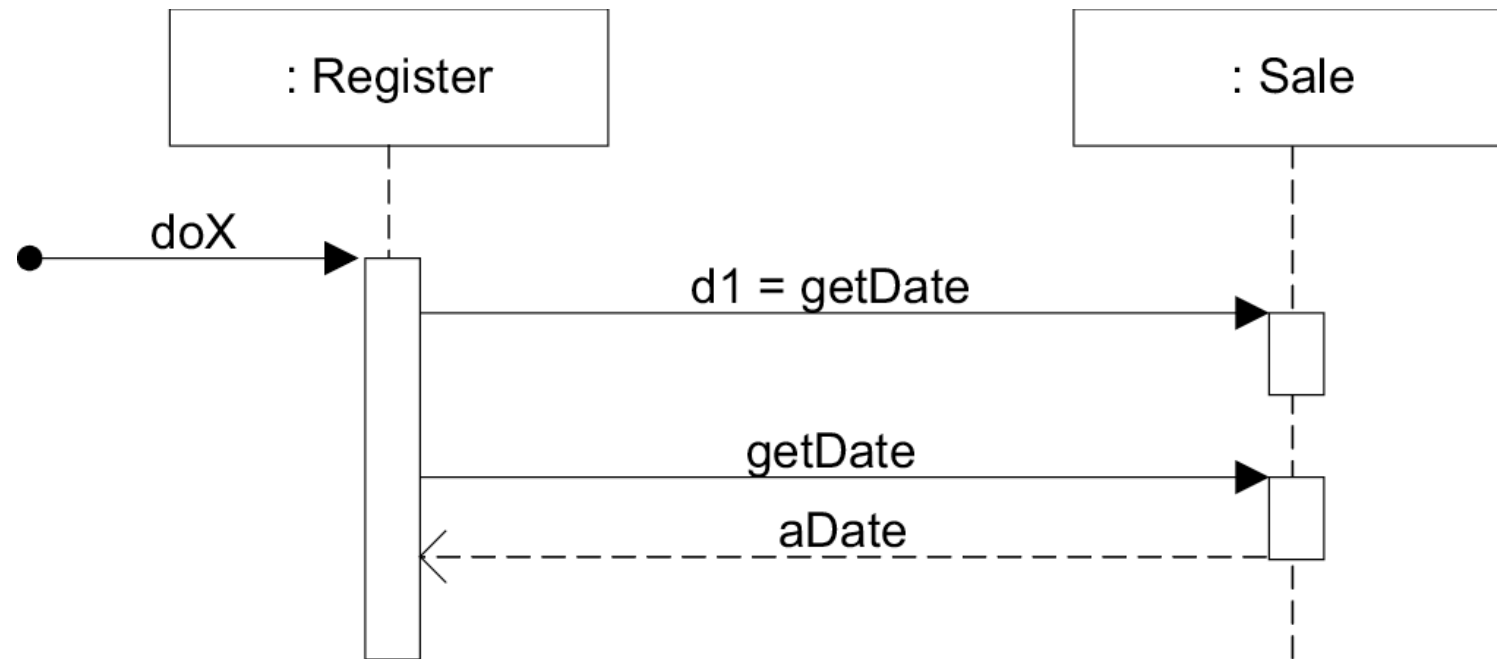
Corpo di un metodo

Diagrammi di sequenza

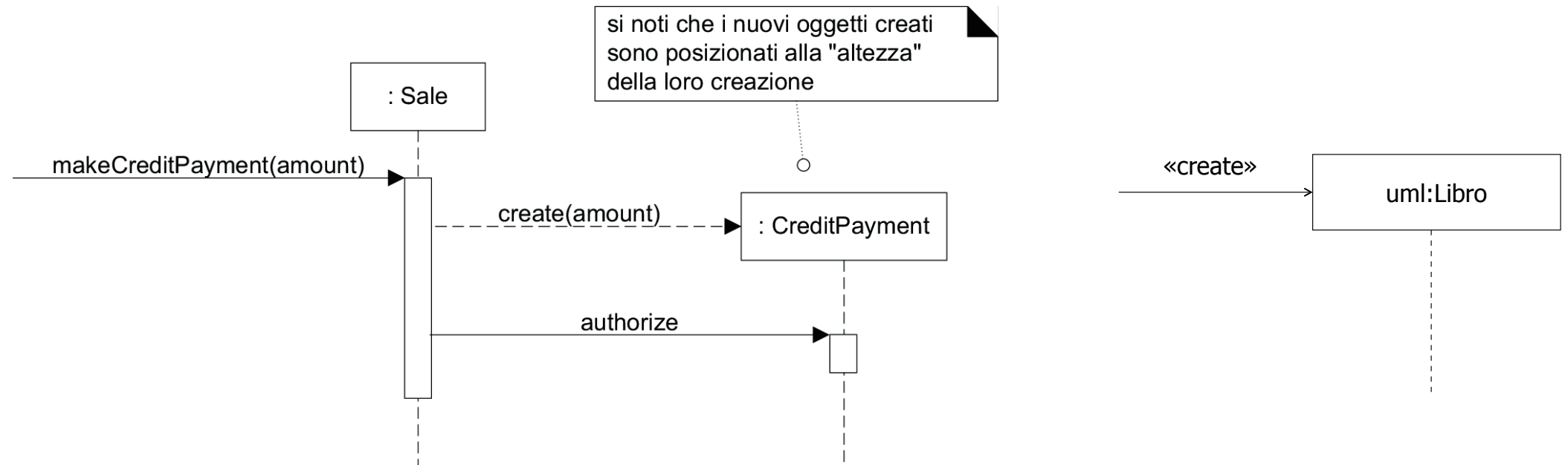


- Tutti i diagrammi di interazione possono essere prefissati da **sd** per indicare il loro tipo
- Le attivazioni indicano quando una linea di vita ha il focus di controllo

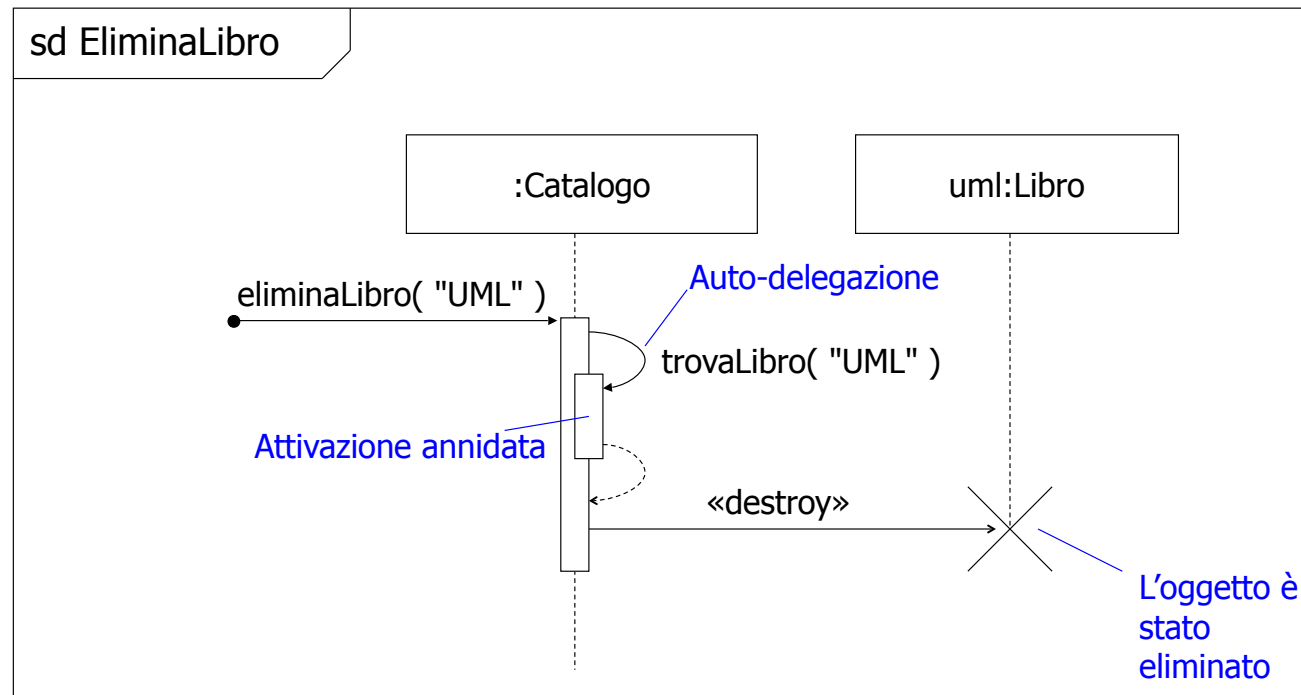
Illustrare le risposte e i ritorni



Creazione di istanza

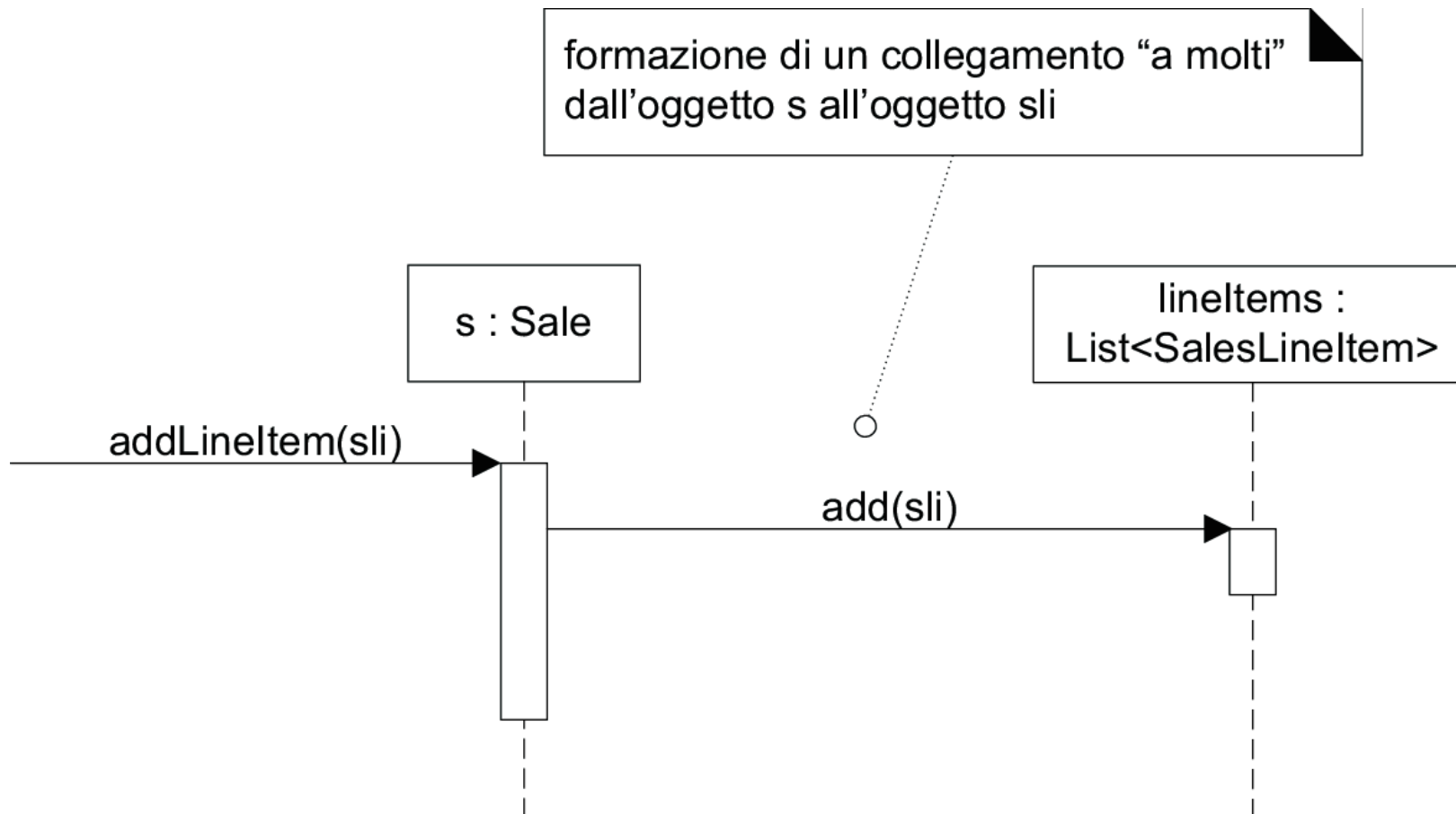


Distruzione di oggetti e auto-delega

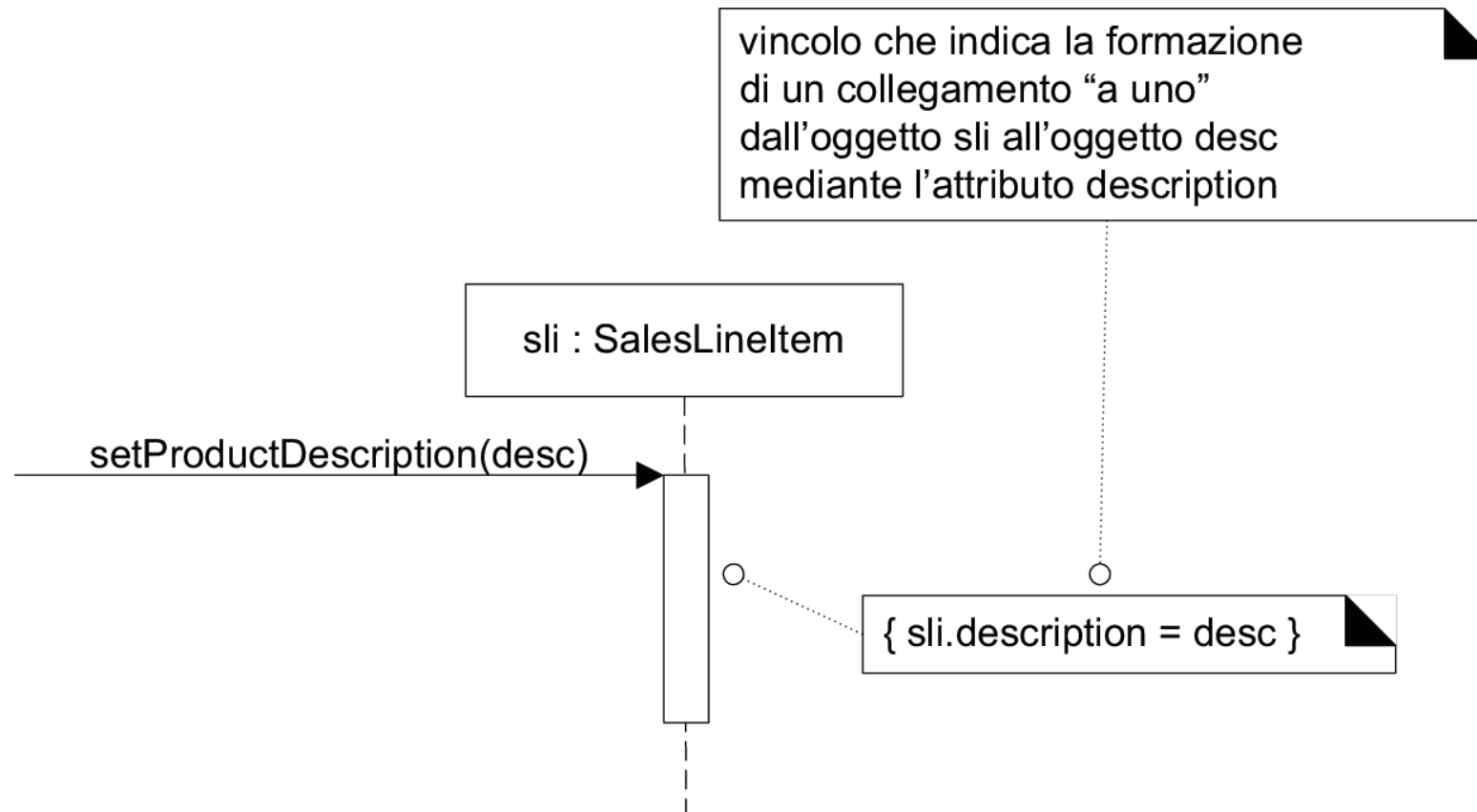


- Auto-delegazione è quando una linea vita invia un messaggio a se stessa
 - *Genera un attivazione annidata*
- Per indicare la distruzione di un oggetto, si termina la linea di vita con una grossa croce

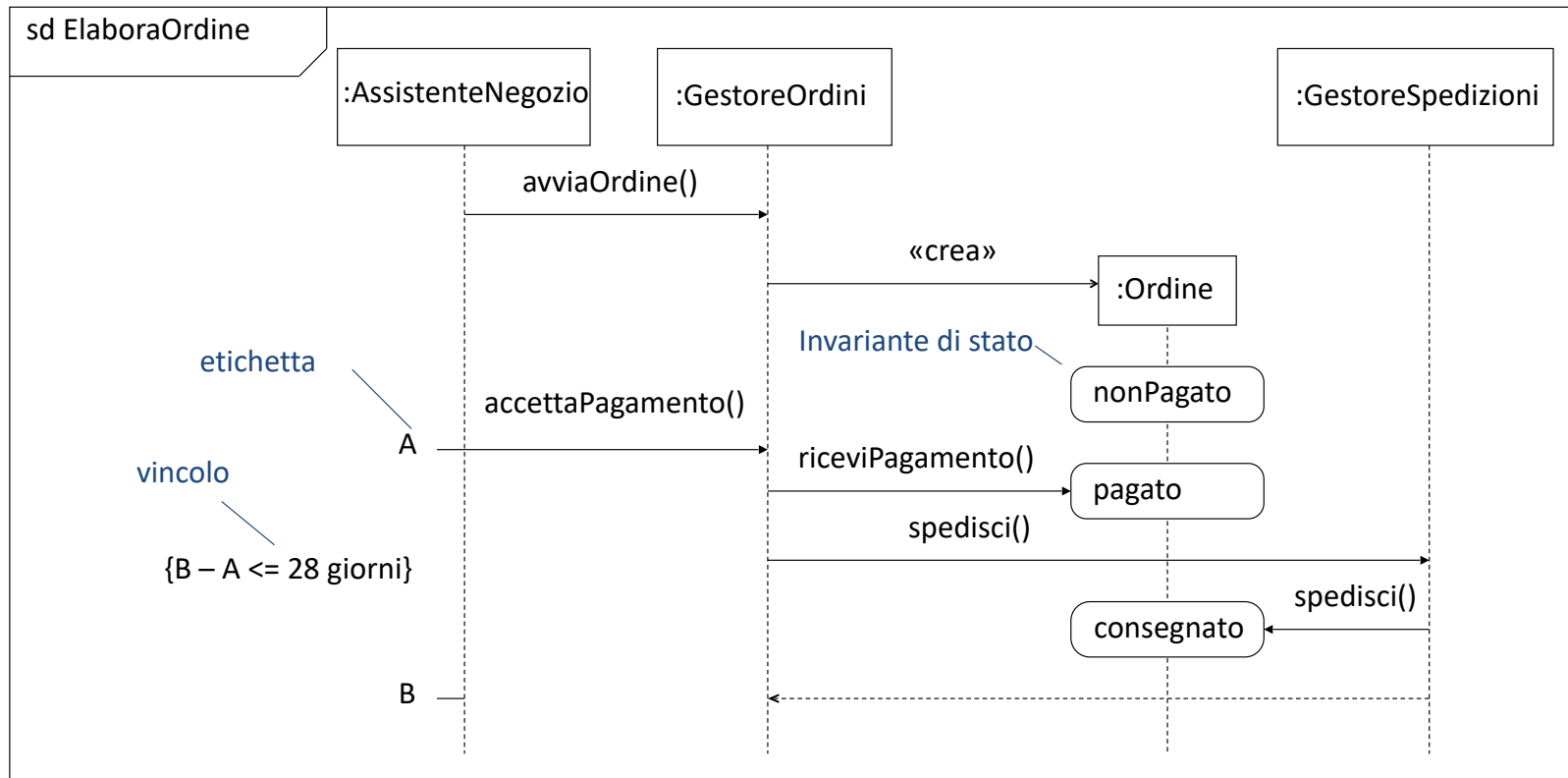
Formazione di un collegamento di un'associazione “a molti”



Formazione di un collegamento di un'associazione "a uno"

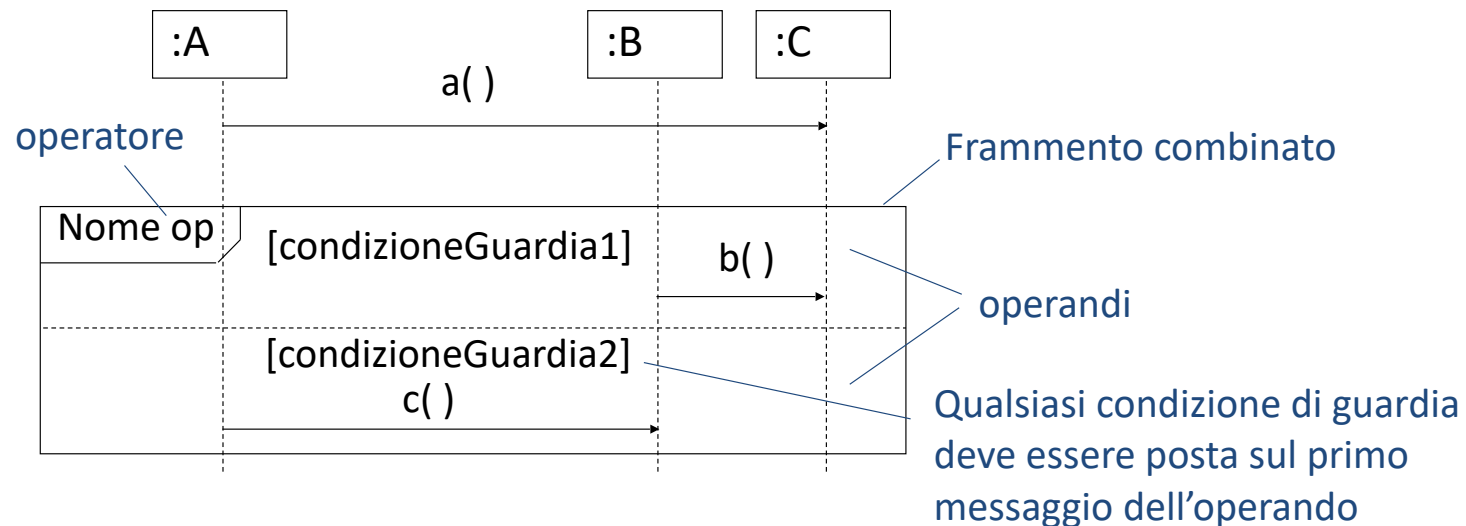


Invarianti di Stato



- Uno stato è definito “una condizione o situazione durante la vita di un oggetto in cui esso soddisfa una condizione, esegue un attività o aspetta un evento” [Rumbaugh]
- Ogni classificatore può avere una macchina a stati che descrive il ciclo di vita delle sue istanze in termini degli stati e degli eventi che causano la transizione tra questi stati

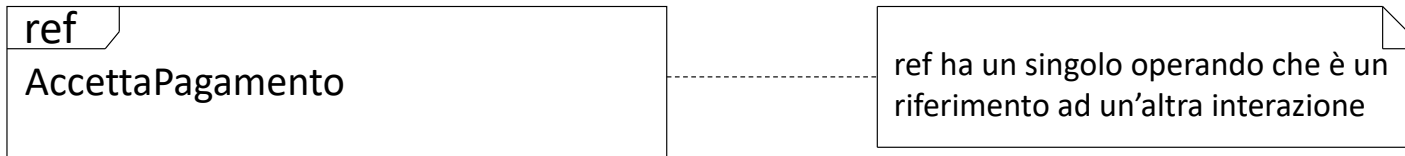
Frammenti Combinati (Frame)



- I diagrammi di sequenza possono essere divisi in aree chiamate frammenti combinati
- I frammenti combinati hanno uno o più operandi
- L'operatore determina come vengono eseguiti i suoi operandi
- Le condizioni di guardia stabiliscono **se** i loro operandi devono essere eseguiti. L'esecuzione occorre se la condizione di guardia è valutata vera

Operatori più usati

operatore	Nome completo	significato
opt	Option	C'è un singolo operando che viene eseguito se la condizione è vera (come if ... then)
alt	Alternatives	Viene eseguito l'operando la cui condizione è vera. Si può usare la parola chiave altrimenti invece di un'espressione Booleana (come select... case)
loop	Loop	Ha una sintassi speciale: loop min, max [condizione] Esegui loop min volte, poi mentre la condizione è vera, esegui loop altre (max-min) volte
break	break	Se la condizione di guardia è vera, viene eseguito l'operando, non il resto dell'interazione in cui è incluso
ref	Reference	Il frammento combinato fa riferimento a un'altra interazione

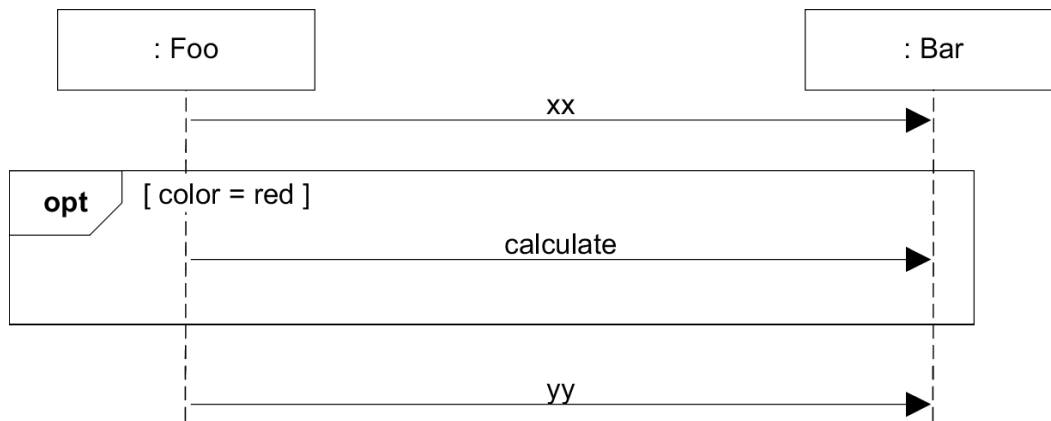


Altri operatori

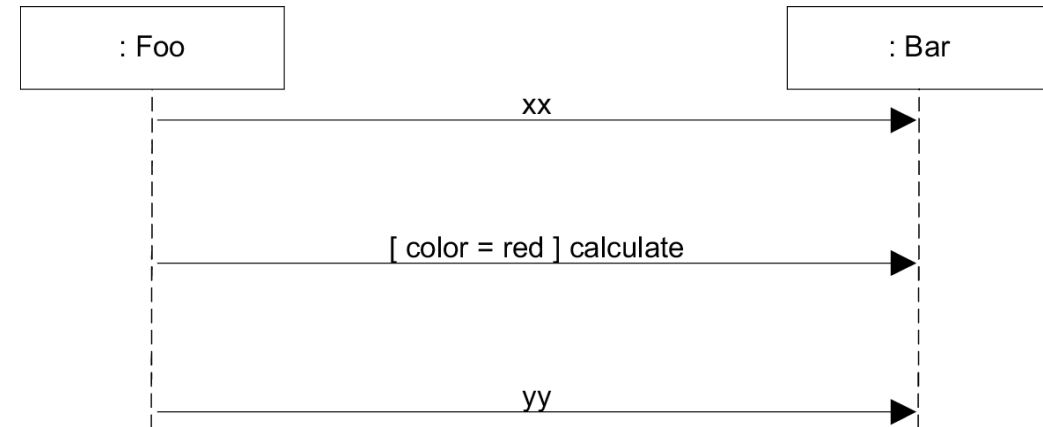
- Questi operatori sono meno comuni

operatore	Nome completo	significato
par	parallel	Tutti gli operandi sono eseguiti in parallelo
seq	weak sequencing	Tutti gli operandi vengono eseguiti in parallelo con il vincolo che gli eventi che arrivano sulla stessa linea di vita da operandi differenti avvengano nella stessa sequenza degli operandi
strict	strict sequencing	Gli operandi vengono eseguiti in sequenza stretta
neg	negative	L'operando mostra interazioni che non devono accadere
critical	critical region	L'operando viene eseguito atomicamente senza interruzione

Messaggi Condizionali: opt



UML 2

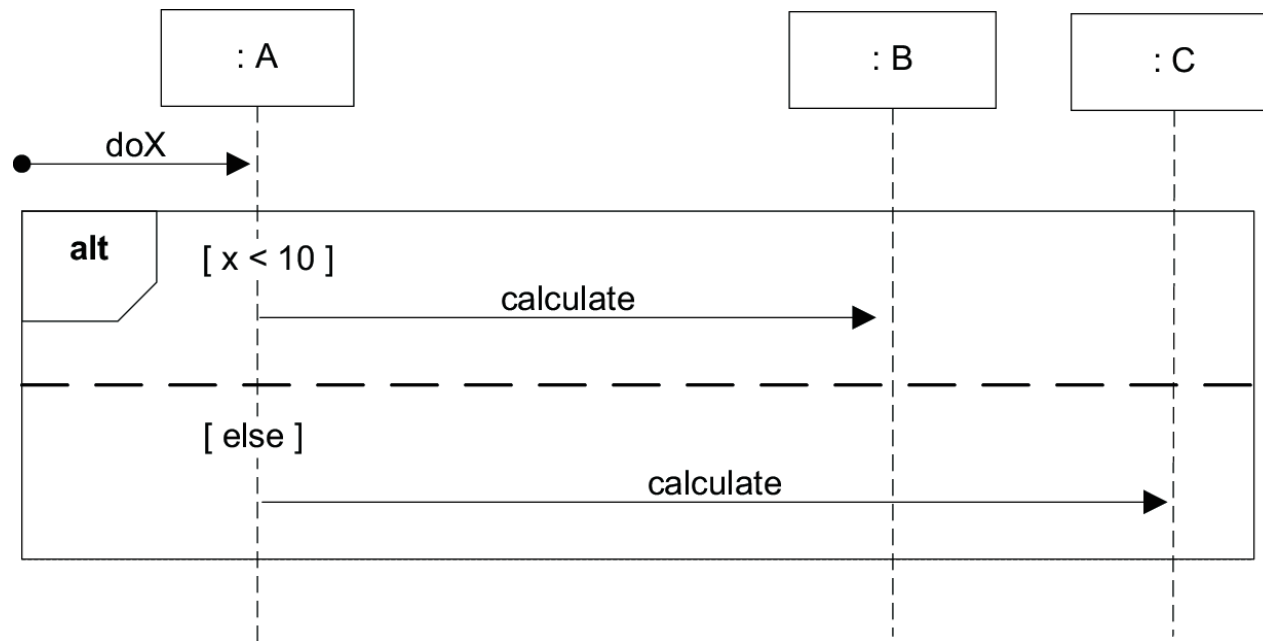


UML 1

opt semantica:

- Il suo unico operando viene eseguito se e solo se la condizione è vera

Messaggi Condizionali mutuamente esclusivi: alt

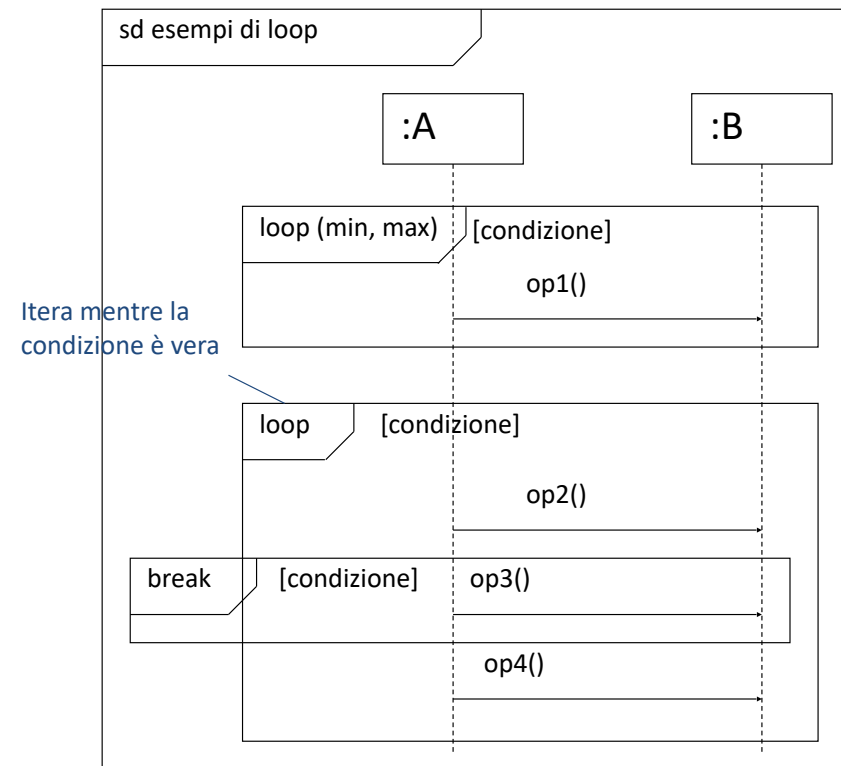


alt semantica:

- Due o più operandi ognuno protetto dalla propria condizione di guardia
- Un operando viene eseguito se la sua condizione è true
- L'operando opzionale con condizione [altrimenti] viene eseguito se nessuna delle altre condizioni è vera

Iterazioni con loop e break

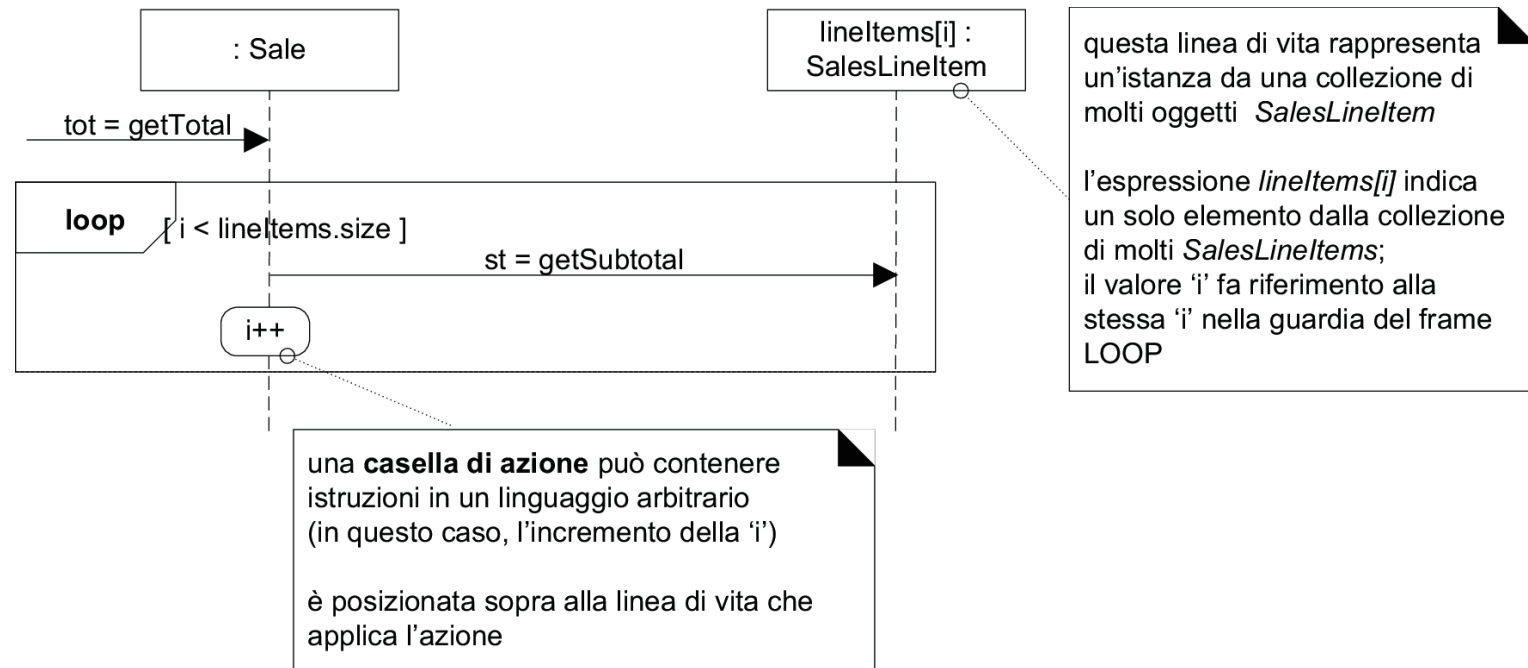
- loop significato:
 - Esegui loop min volte, poi mentre la condizione è vera, esegui loop altre (max-min) volte
- loop sintassi
 - Un ciclo (loop) senza max, min o una condizione è un ciclo infinito
 - Se viene specificato dolo min, allora si considerare max=min
 - La condizione può essere
 - Un espressione booleana
 - Un testo purché il suo significato sia chiaro!
- Il Break indica in quale condizione il loop viene interrotto e che cosa succede inseguito:
 - Il resto del ciclo non viene eseguito dopo l'interruzione (break)
 - Il frammento combinato break è logicamente esterno al ciclo e si deve disegnare sovrapponendolo al loop



Idiomi più comuni

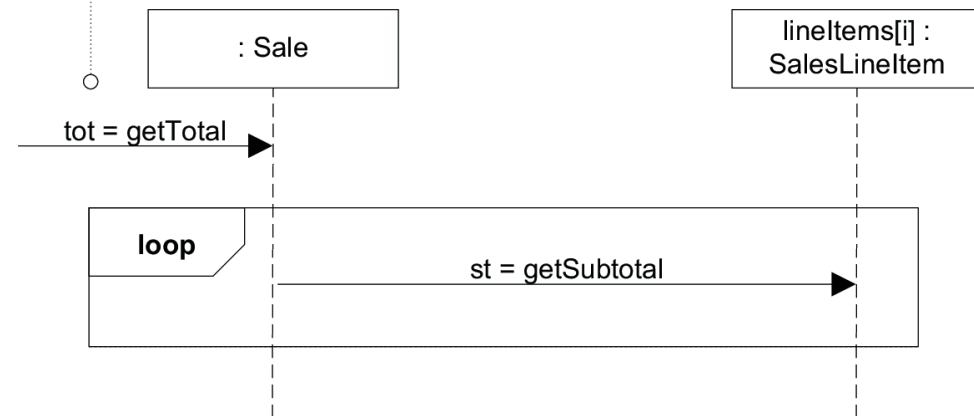
Tipo di ciclo	significato	Espressione del ciclo
While(true) {body}	Continuare il ciclo per sempre	loop *
for i = 1 to n {body}	ripetere n volte	loop n
While(espressioneBooleana) {body}	Ripetere mentre espressioneBooleana è vera	loop [espressioneBooleana]
repeat {body} while(espressioneBooleana)	Eseguire una volta poi ripetere mentre espressioneBooleana è vera	loop 1, * [espressioneBooleana]
forEach oggetto della collezione {body}	Eseguire il corpo del ciclo una volta per ogni oggetto di una collezione di oggetti	loop [per ogni oggetto in CollezioneDiOggetti]
forEach oggetto della classe {body}	Eseguire il corpo del ciclo una volta per ogni oggetto di una particolare classe	loop [per ogni oggetto in NomeClasse]

Iterazione su una collezione



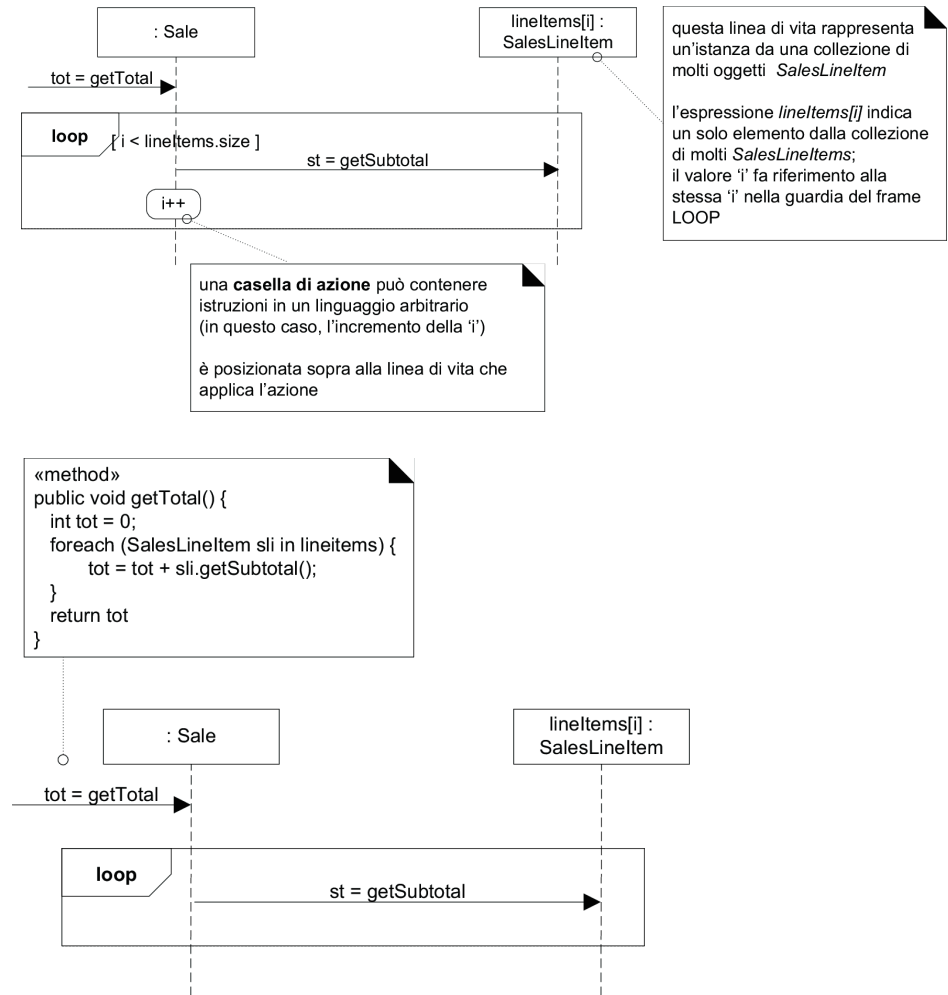
Iterazione su una collezione

```
«method»  
public void getTotal() {  
    int tot = 0;  
    foreach (SalesLineItem sli in lineitems) {  
        tot = tot + sli.getSubtotal();  
    }  
    return tot  
}
```

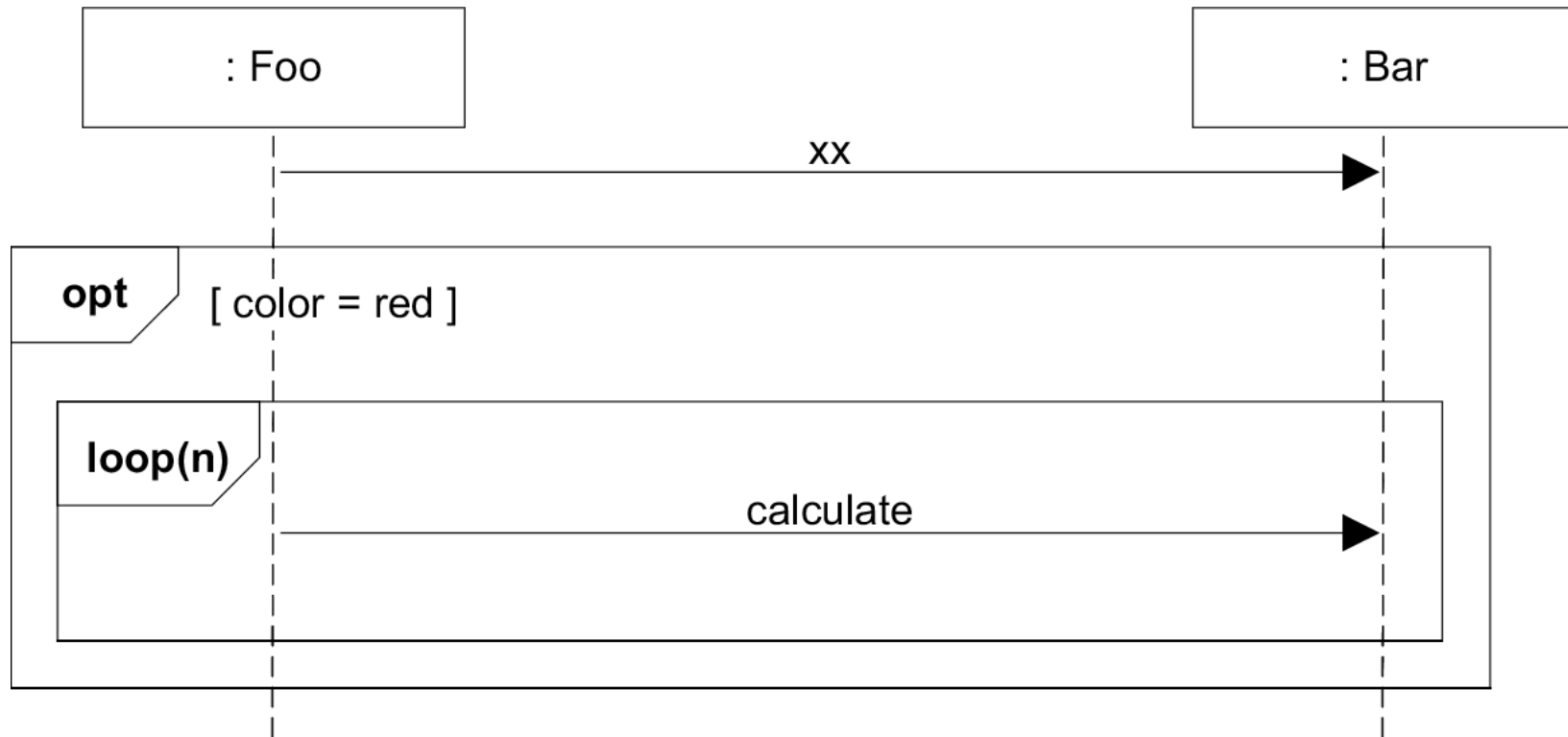


Iterazione su una collezione

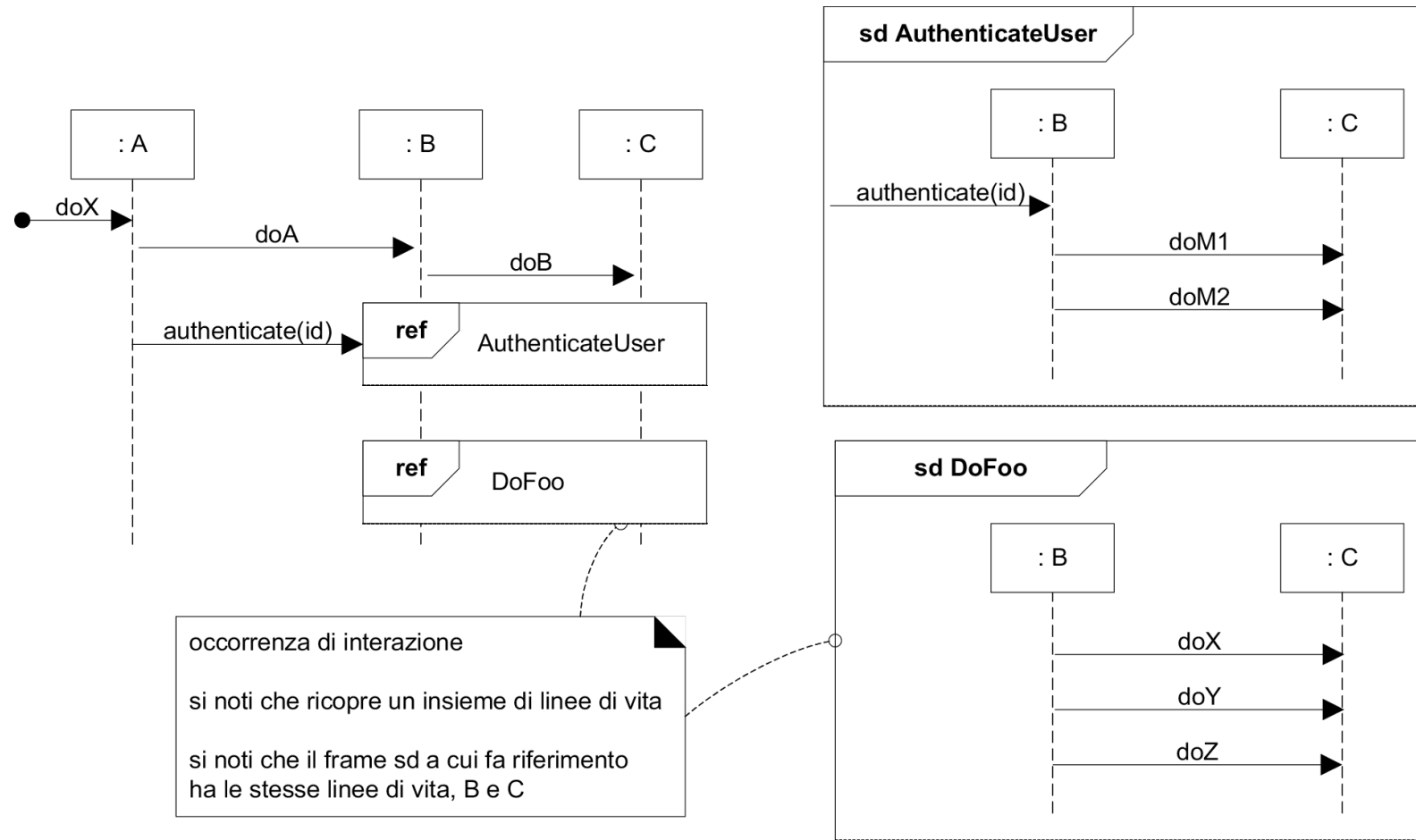
```
public class Sale {  
    private List<SalesLineItem> lineItems;  
    ...  
    public Money getTotal() {  
        Money total = new Money();  
        Money subtotal;  
        for (SalesLineItem lineItem : lineItems) {  
            subtotal = lineItem.getSubtotal();  
            total.add(subtotal);  
        }  
        return total;  
    }  
    ...  
}
```



Annidamento di frame

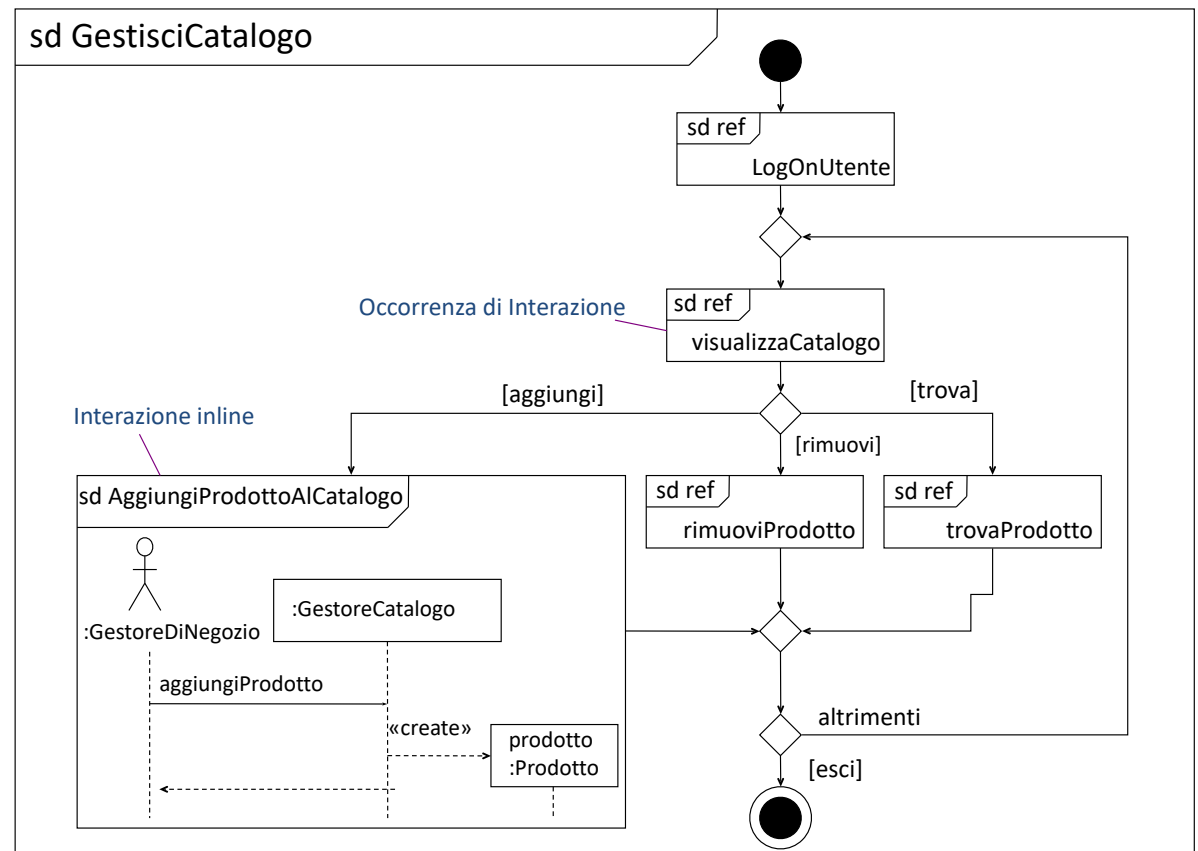


Come correlare i diagrammi

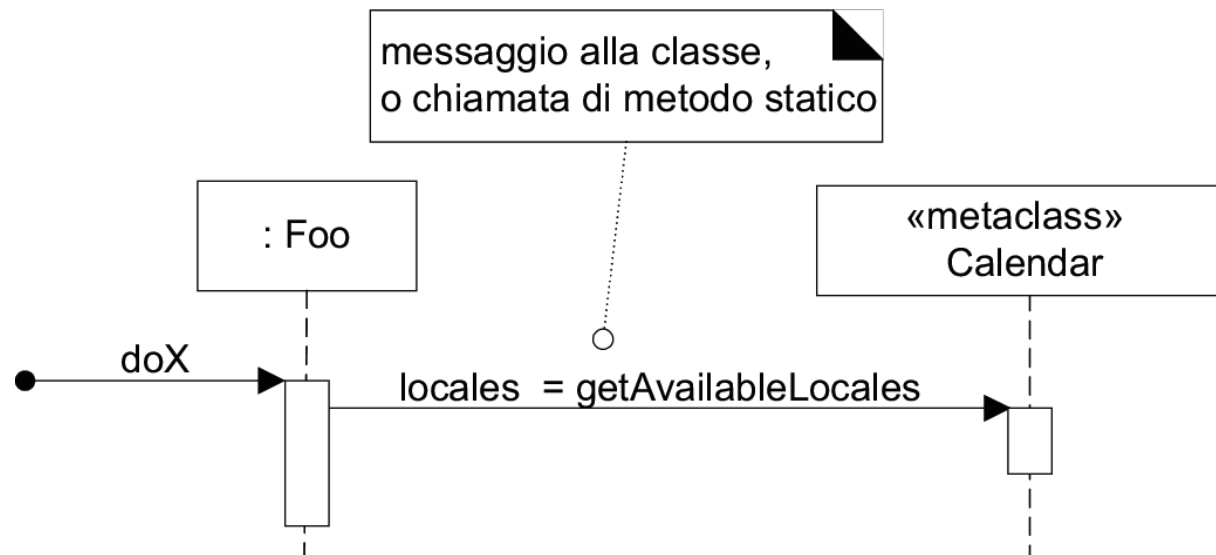


Diagrammi di Interazione Generale

- Modellano il flusso di controllo tra interazioni ad alto livello
- Mostrano le interazioni e le occorrenze di interazioni
- Hanno la sintassi dei diagrammi di attività

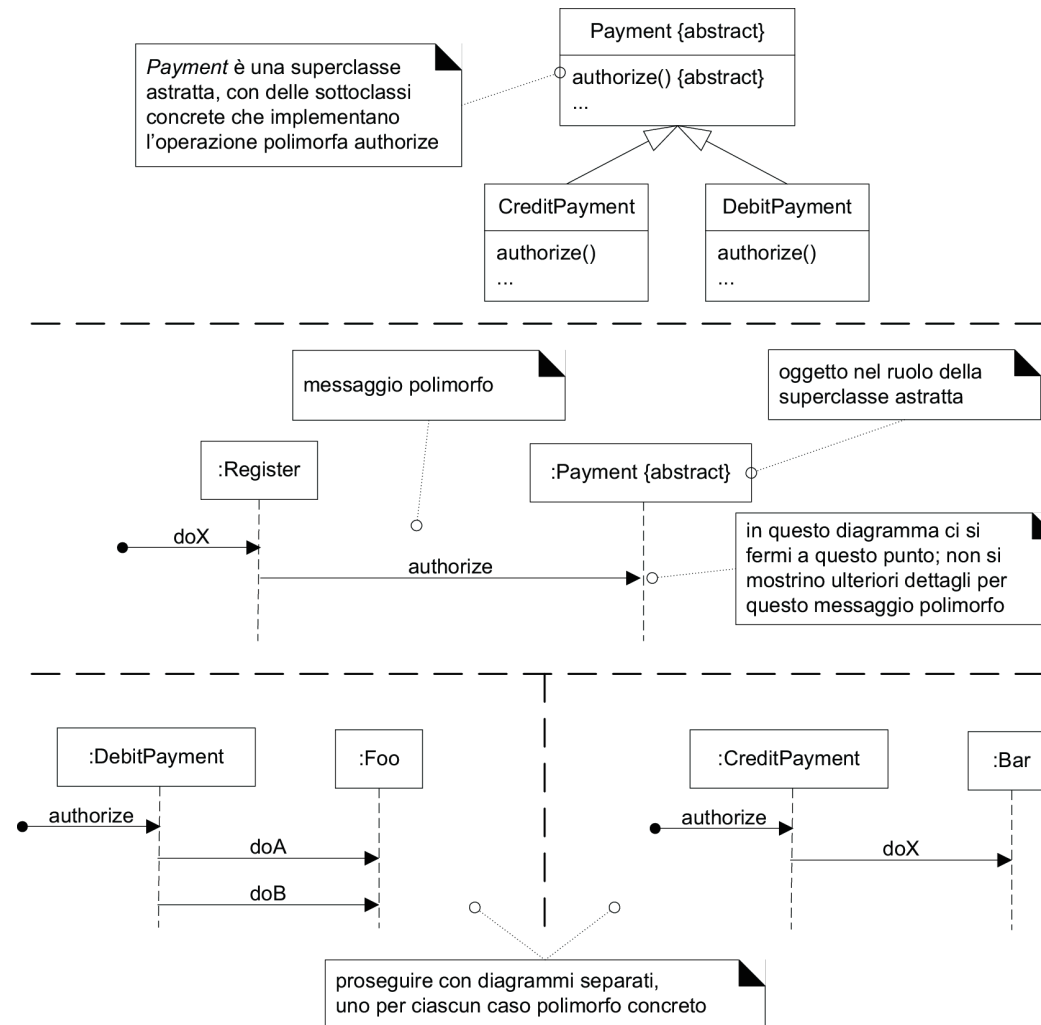


Messaggi alle classi per invocare metodi statici



```
// chiamata di metodo statico sulla classe Calendar
Locale[] locales = Calendar.getAvailableLocales();
```

Messaggi e casi polimorfi



Chiamate asincrone e oggetti attivi

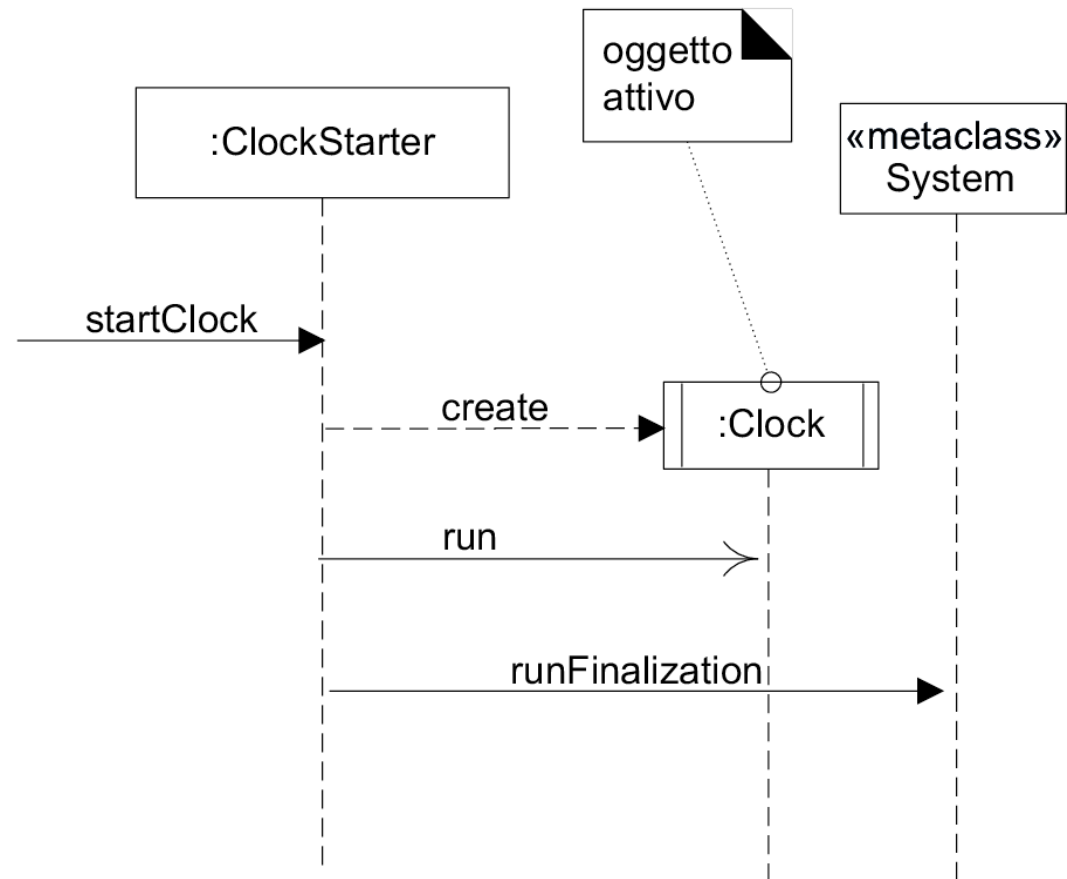
una freccia non piena indica una **chiamata asincrona** (diversamente da una freccia piena, che invece rappresenta una più comune chiamata sincrona)

in Java, per esempio, una chiamata asincrona può avvenire come segue:

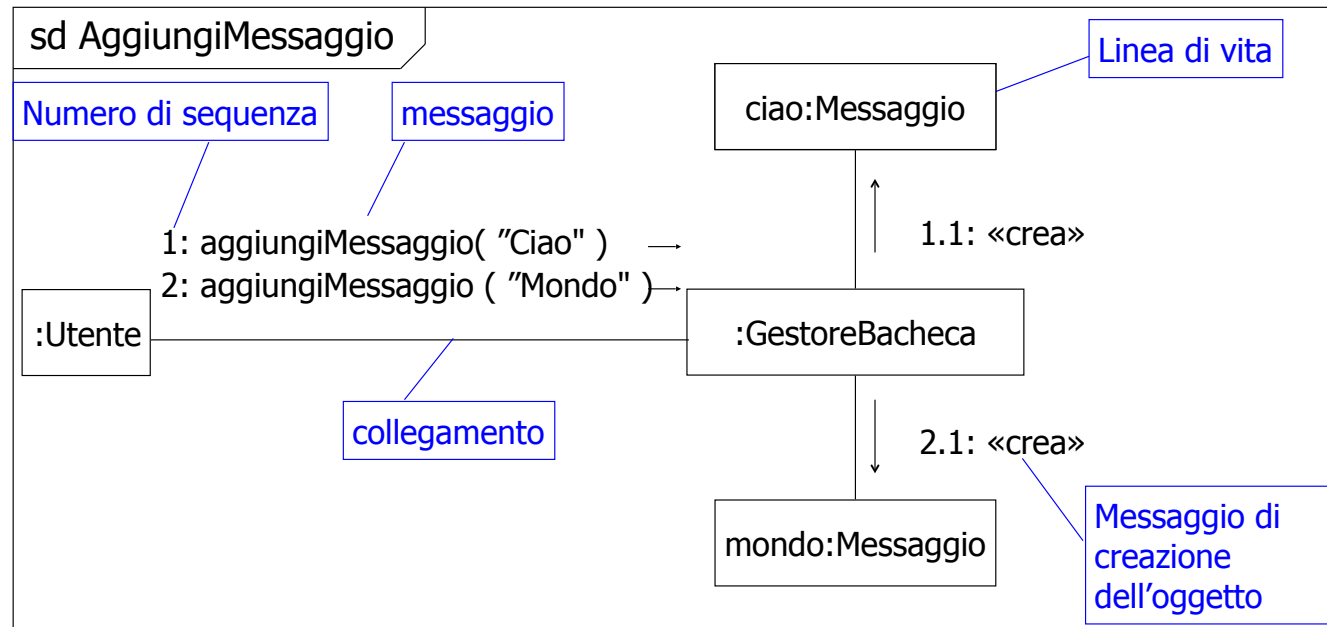
```
// Clock implementa l'interfaccia Runnable  
Thread t = new Thread( new Clock() );  
t.start();
```

la chiamata di *start* implica poi una chiamata asincrona del metodo *run* dell'oggetto *Runnable* (*Clock*)

per semplificare il diagramma, l'oggetto *Thread* e il messaggio *start* si possono evitare (sono un “overhead” standard); al contrario, la chiamata asincrona è implicata dai dettagli essenziali della creazione di *Clock* e dal messaggio *run*

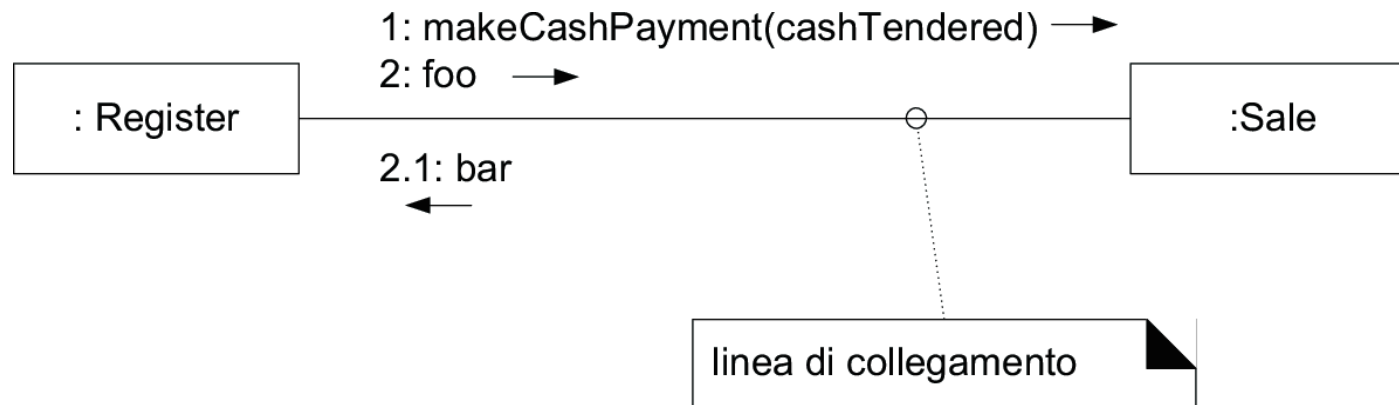


Diagrammi di comunicazione



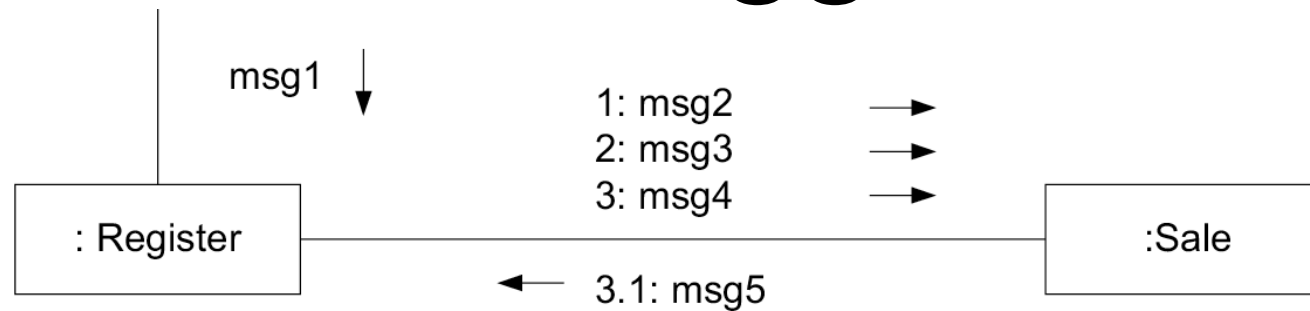
- I diagrammi di comunicazione enfatizzano gli aspetti strutturali di un'interazione – mostrando come si collegano le linee di vita

Collegamento



- Un collegamento (link) è un percorso di connessione tra due oggetti
 - *indica l'esistenza di qualche forma di navigabilità e visibilità tra gli oggetti*

Messaggi

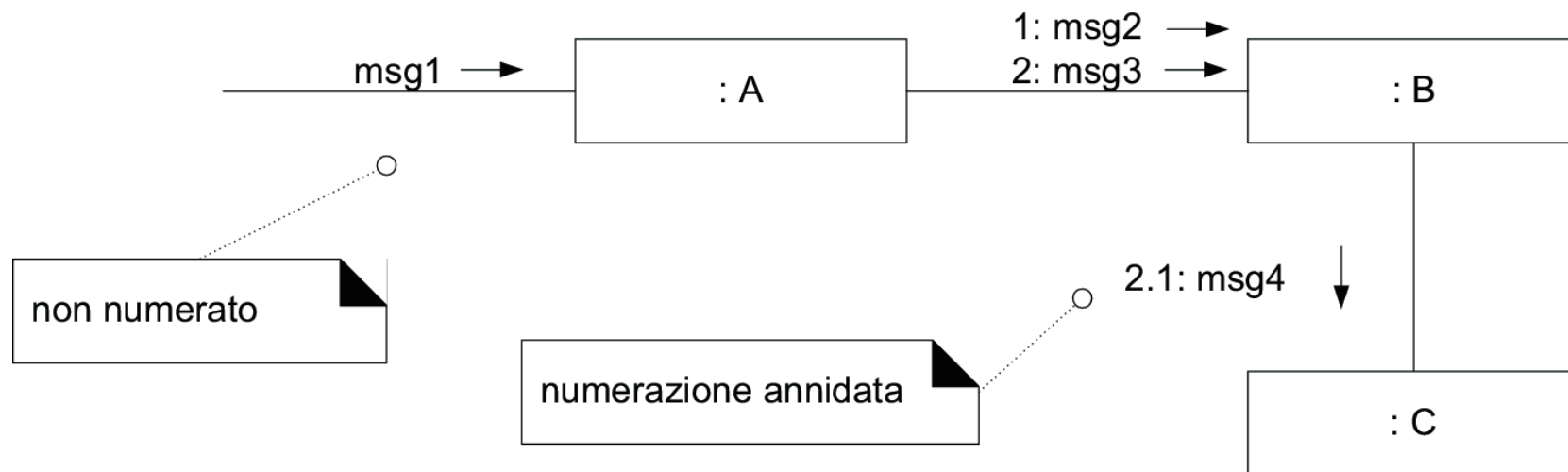


tutti i messaggi scorrono sullo stesso collegamento

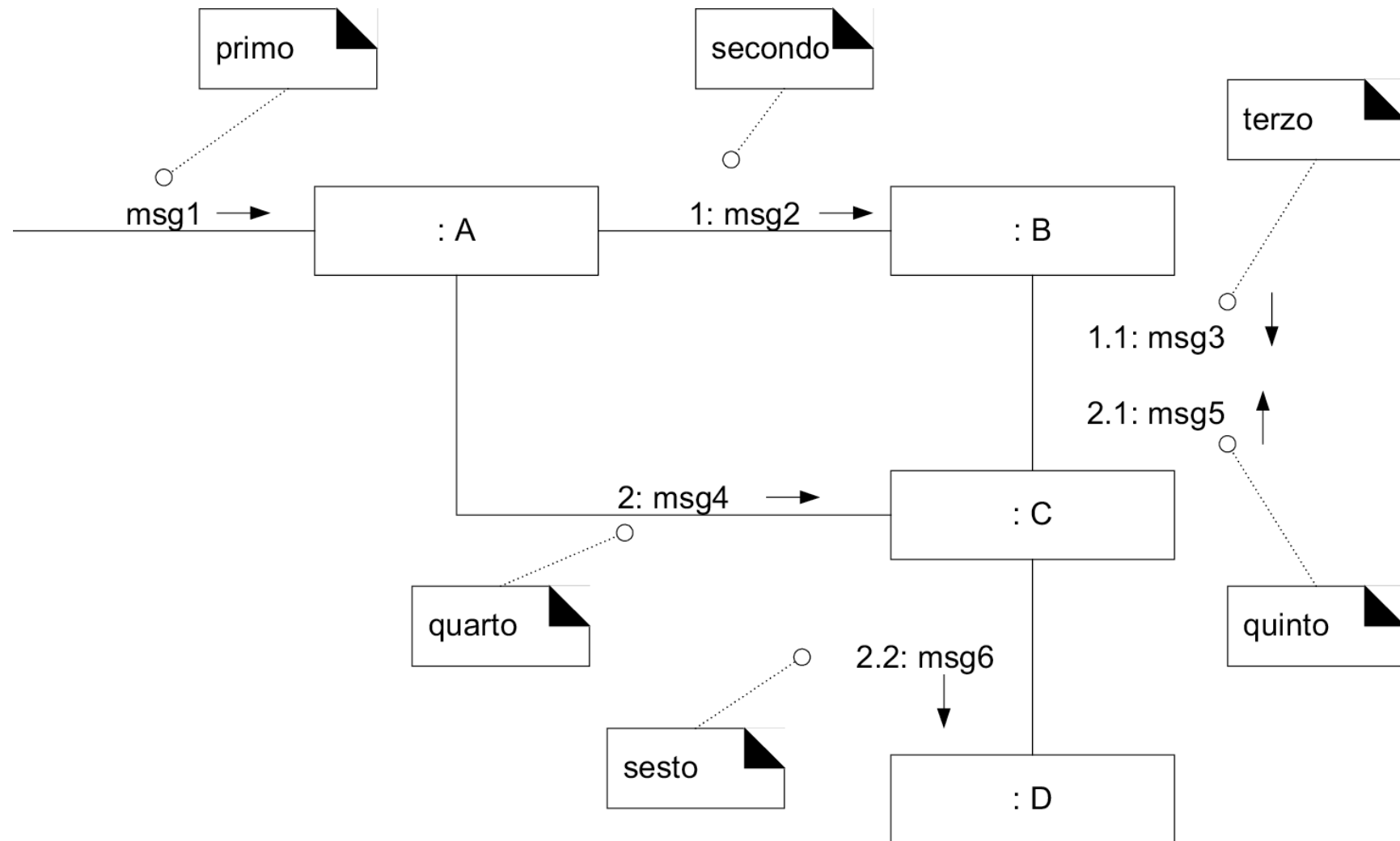
Linea guida

non numerate il messaggio trovato iniziale, ciò semplifica la numerazione complessiva

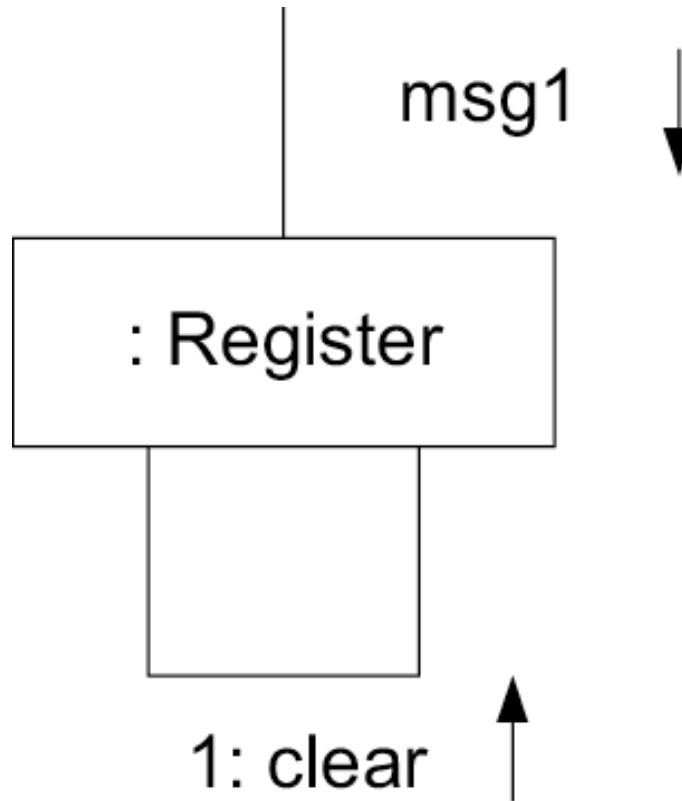
Numerazione di una sequenza



Numerazione di una sequenza complessa



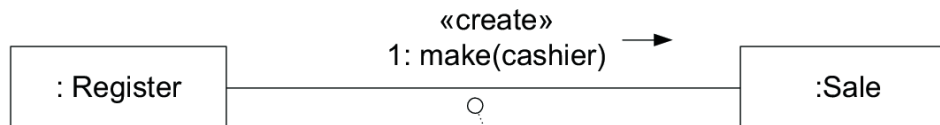
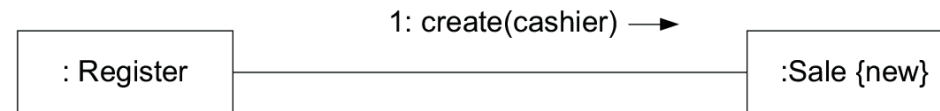
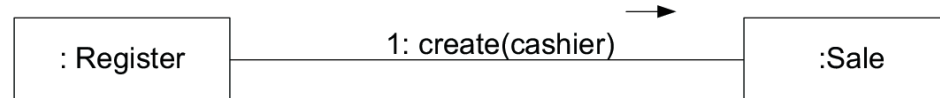
Auto-delega: messaggi a “self” o “this”



Creazione di istanze

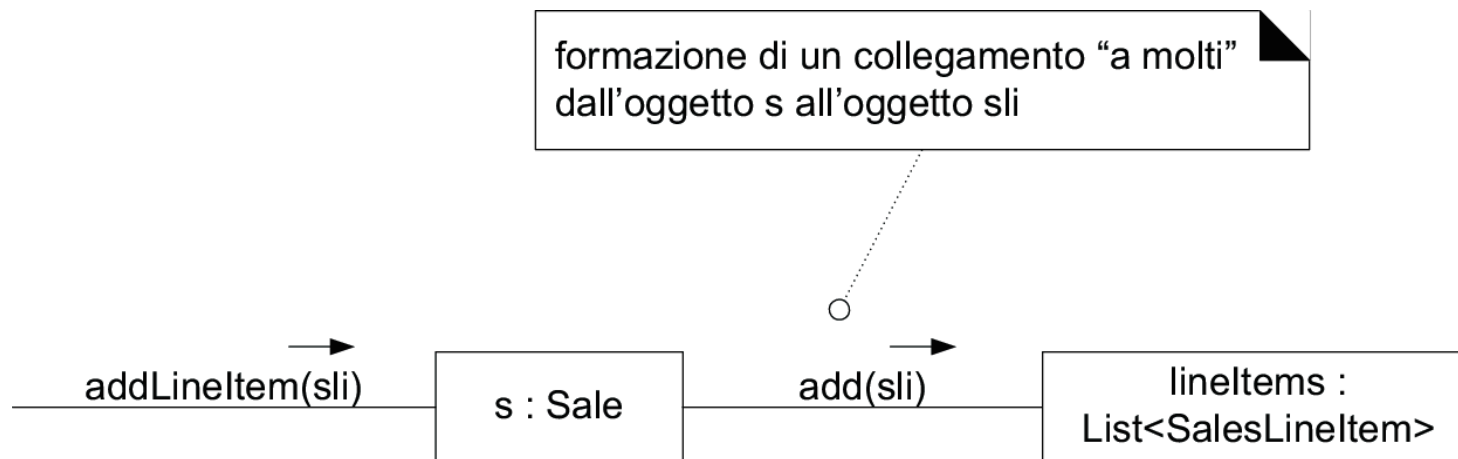
tre modi per mostrare la creazione
in un diagramma di comunicazione

un messaggio create con dei parametri opzionali di
inizializzazione; questo verrà normalmente interpretato
come una chiamata del costruttore

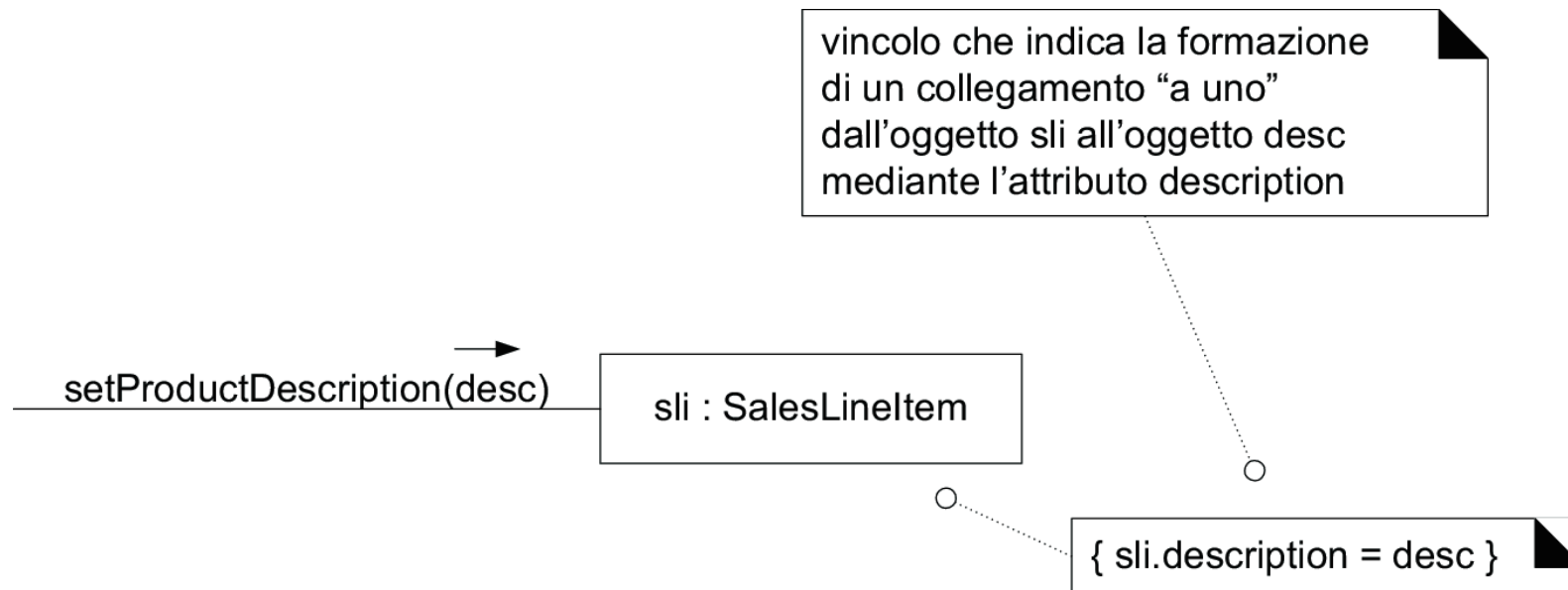


se viene usato un nome di messaggio non ovvio
per la creazione, al messaggio può essere aggiunto
per chiarezza uno stereotipo «create»

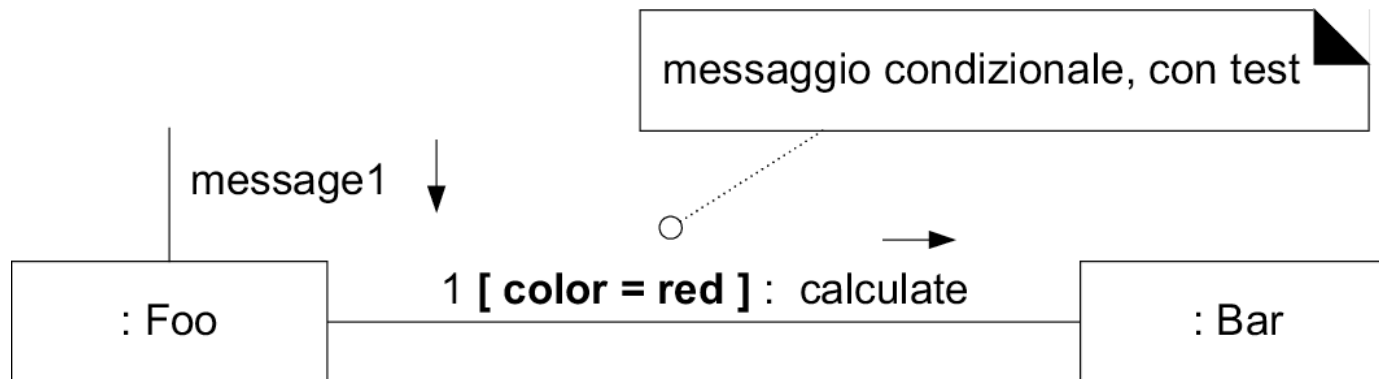
Formazione di un collegamento di un'associazione “a molti”



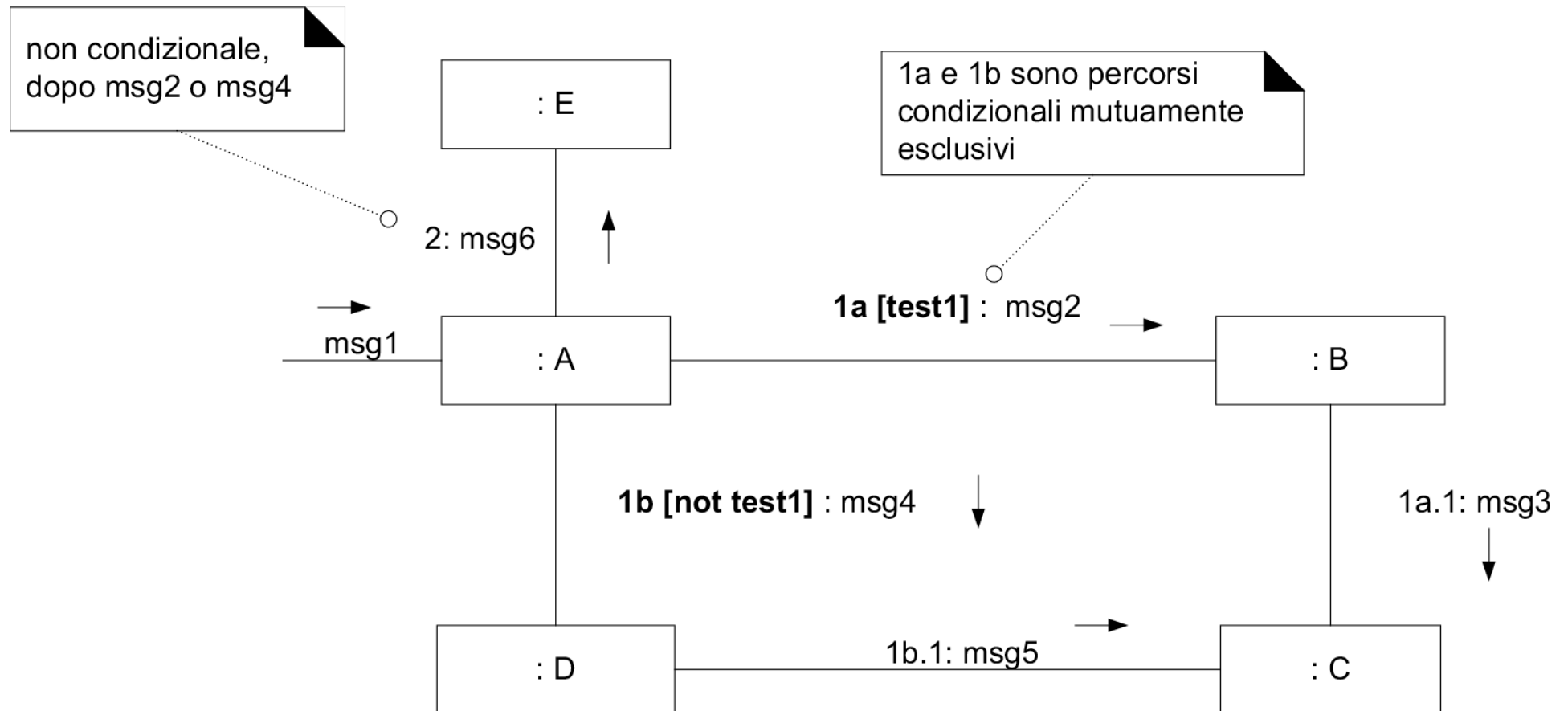
Formazione di un collegamento di un'associazione "a uno"



Messaggi condizionali

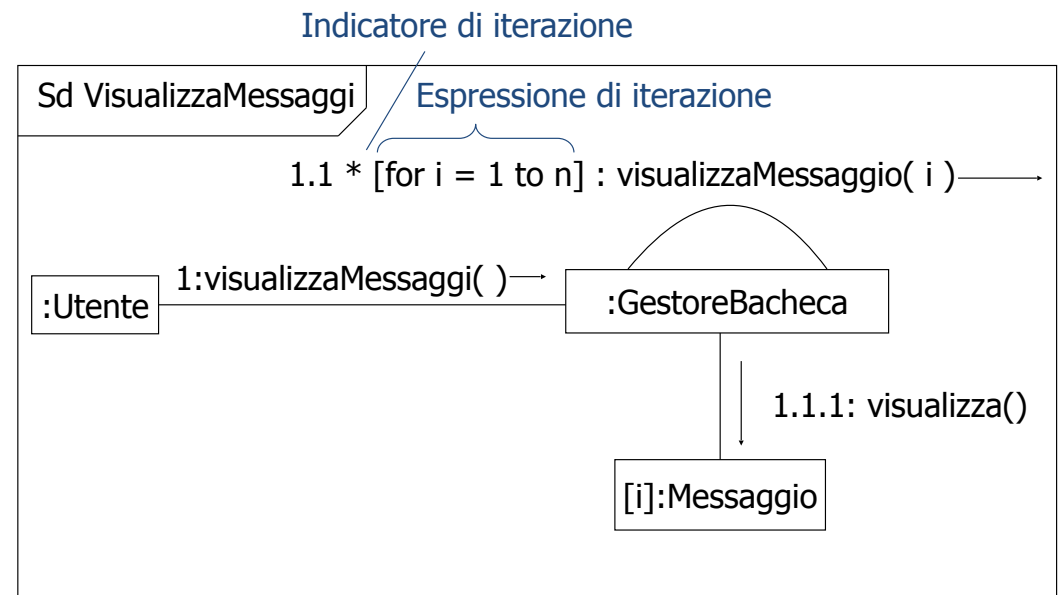


Percorsi condizionali mutuamente esclusivi

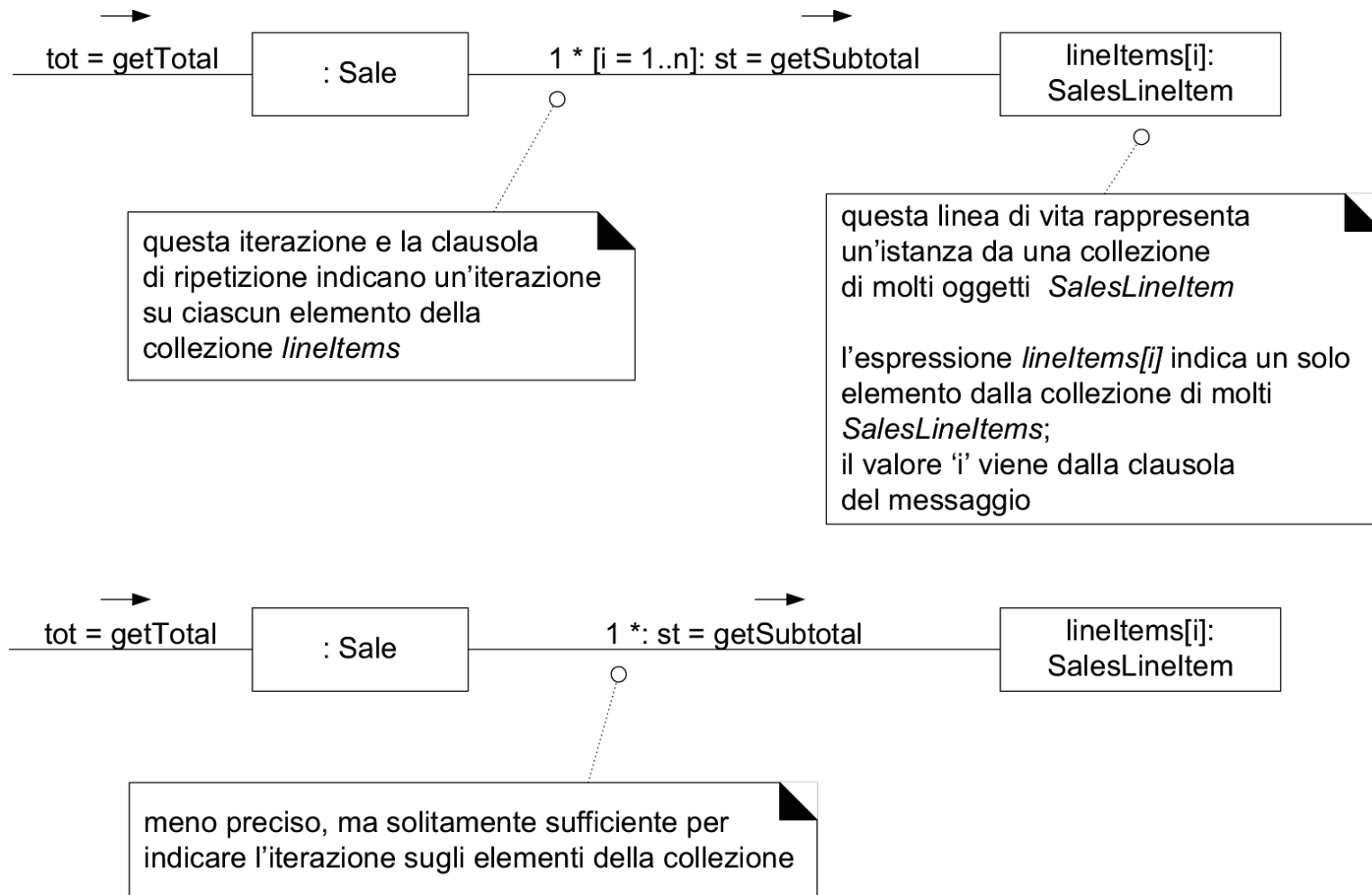


Iterazione o ciclo

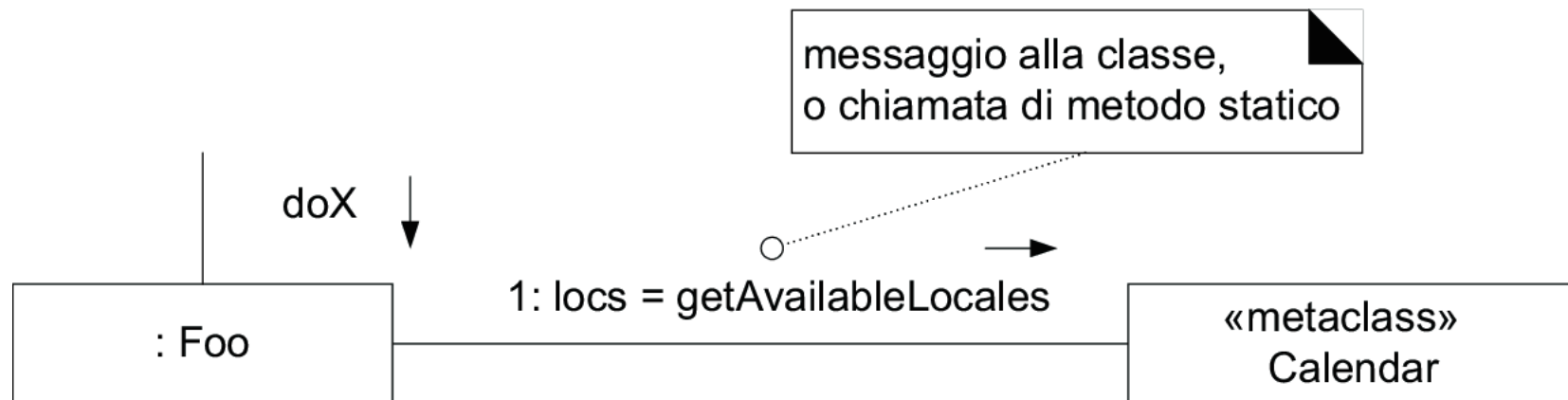
- L'iterazione è mostrata dall'utilizzo dell'indicatore di iterazione (*) e un'espressione di iterazione
 - *UML 2 non stabilisce alcuna sintassi particolare per l'espressioni di iterazione*
 - *L'uso di codice o pseudo-codice è una buona scelta*
- Se si vuole indicare che i messaggi sono tutti eseguiti in parallelo, si deve utilizzare l'indicatore parallelo di iterazione, ***//**



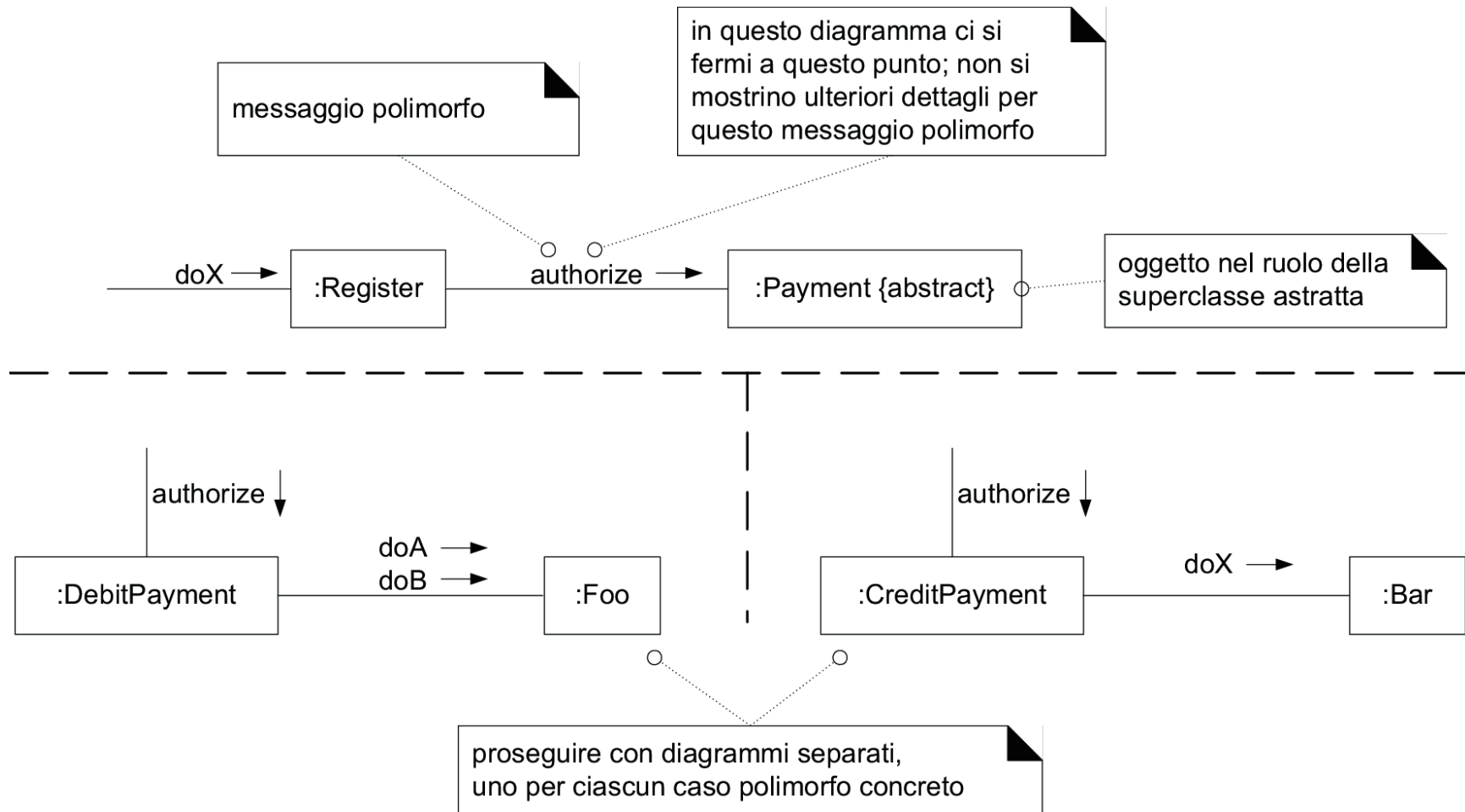
Iterazione su una collezione



Messaggi a una classe per invocare i metodi statici



Casi polimorfi con i diagrammi di comunicazione



Chiamate Asincrone

